

Natural Language Processing

An Introduction

Abdelbacet Mhamdi

2026-05-07

MT @ ISET Bizerte

1. NLP Fundamentals	2
2. Introduction to Regular Expressions (Regex)	12
3. Text Tokenization	42
4. Text Processing and Visualization	78
5. Word Embedding	132
6. Deep Learning for NLP	147

1. NLP Fundamentals

1.1 Terminology

Natural Language Processing (NLP) is a subfield of **AI** that focuses on the interaction between computers and linguistics. It involves the development of algorithms and models that enable machines to understand, interpret, and generate human language in a way that is both meaningful and useful.

Examples: *Text analysis, sentiment analysis, machine translation, chatbots.*

A **corpus** is a large, structured collection of texts used for linguistic analysis and model training. **Corpora** provide diverse language samples that help algorithms learn patterns, relationships, and semantic meanings.

Examples: *Wikipedia dumps, news articles, twitter data.*

A **vocabulary** is the complete set of unique words or tokens recognized by a language model. The **vocabulary** defines the model's linguistic boundaries and is typically extracted from training data with frequency thresholds.

1.1 Terminology

Tip

dictionary, word list, token collection.

A **document** refers to a discrete text unit processed as a single entity. **Documents** can vary greatly in length and structure but serve as the fundamental unit for many text analysis tasks.

Examples: *emails, articles, tweets, books.*

1.2 Linguistic Levels

Language can be analyzed at five progressive levels, each building on the previous:

Phonology → **Morphology** → **Syntax** → **Semantics** → **Pragmatics**
(sounds) (words) (grammar) (meaning) (intent)

Each level adds a richer layer of linguistic understanding — from raw sound to human intent.

1.2 Linguistic Levels

1.2.1 Level 1 — Phonology

The study of **sound systems** in language — how sounds are organized, patterned, and perceived.

- **Phonemes** — smallest units of sound (/p/, /b/, /t/)
- **Allophones** — sound variants of a phoneme
- **Prosody** — rhythm, stress, and intonation
- **Minimal pairs** — *bat* vs *cat* vs *hat*

Task	Example
ASR	Speech-to-text
TTS	Text-to-speech
G2P	Grapheme-to-phoneme
Speaker ID	Voice recognition

“cat” → /k/ /æ/ /t/

1.2 Linguistic Levels

1.2.2 Level 2 — Morphology

The study of **word structure** — how words are formed from smaller meaningful units called morphemes.

- **Morpheme** — smallest unit of meaning
- **Root / Stem** — *play* in *playing*
- **Affixes** — prefixes (*un-*), suffixes (*-ing*)
- **Inflection** — *walk* → *walked*, *walks*
- **Derivation** — *happy* → *unhappy* → *unhappiness*

Task	Example
Stemming	<i>running</i> → <i>run</i>
Lemmatization	<i>better</i> → <i>good</i>
Tokenization	Subword splitting
Morphological analysis	POS tagging

“unhappiness” → *un-* + *happy* + *-ness*

1.2 Linguistic Levels

1.2.3 Level 3 — Syntax

The study of **sentence structure** — rules governing how words combine into grammatically correct sentences.

- **POS Tagging** — noun, verb, adjective...
- **Parse Tree** — hierarchical sentence structure
- **Dependency Parsing** — grammatical relations
- **Constituency** — grouping phrases (NP, VP)
- **Grammar rules** — subject-verb agreement

Task	Example
POS Tagging	<i>runs</i> → VERB
Parsing	Constituency trees
Dep. Parsing	subject-of, object-of
Grammar Check	Error correction

[NP The cat] [VP [V sat] [PP on the mat]]

1.2 Linguistic Levels

1.2.4 Level 4 — Semantics

The study of **literal meaning** — what words, phrases, and sentences actually denote.

- **Lexical semantics** — word meaning
- **WSD** — polysemy resolution (*bank*)
- **Semantic roles** — agent, patient, theme
- **Entailment** — “*dog*” entails “*animal*”
- **Distributional semantics** — meaning from context

Task	Example
Word Embeddings	Word2Vec, GloVe
NER	Person / Place / Org
STS	Sentence similarity
Relation Extraction	<i>X founded Y</i>

“He went to the bank” → river or finance?

1.2 Linguistic Levels

1.2.5 Level 5 — Pragmatics

The study of **intended meaning** — how context, speaker, and situation shape interpretation beyond literal words.

- **Speech acts** — requests, promises, assertions
- **Implicature** — implied but unstated meaning
- **Presupposition** — embedded assumptions
- **Co-reference** — “*She*” refers to “*Alice*”
- **Discourse coherence** — how sentences connect

Task	Example
Intent Detection	Chatbot NLU
Co-ref Resolution	Pronoun linking
Dialogue Systems	State tracking
Sarcasm Detection	“ <i>Oh great</i> ”

“*Can you pass the salt?*” → not a question about ability — it’s a request

1.3 Summary at a Glance

Level	Focus	Core Concept	NLP Application
Phonology	Sounds	Phonemes, prosody	ASR, TTS
Morphology	Words	Morphemes, affixes	Stemming, lemmatization
Syntax	Grammar	Parse trees, POS	Parsing, tagging
Semantics	Meaning	Word sense, embeddings	NER, STS, WSD
Pragmatics	Intent	Speech acts, implicature	Dialogue, chatbots

Modern LLMs implicitly learn **all five levels** from large corpora — from raw subword tokens to nuanced conversational intent.

2. Introduction to Regular Expressions (Regex)

2.1 What are Regular Expressions?

Regular expressions are powerful patterns used to match, search, and manipulate text strings. They provide a standardized way to describe search patterns in text, making them an essential tool in programming, text processing, and data validation.

2.2 Core Concepts

2.2.1 Pattern Matching

A regex pattern is a sequence of characters that defines a search pattern. These patterns can be:

- Literal characters that match themselves;
- Special characters (metacharacters) with special meanings;
- Combinations of both.

2.2 Core Concepts

2.2.2 Basic Metacharacters

Metacharacter	Description	Example
.	Any character except newline	a.c matches “abc”, “a1c”, “a@c”
^	Matches start of string	^Hello matches “Hello World”
\$	Matches end of string	world\$ matches “Hello world”
*	Matches 0 or more occurrences	ab*c matches “ac”, “abc”, “abbc”
+	Matches 1 or more occurrences	ab+c matches “abc”, “abbc”, “abbc”
?	Matches 0 or 1 occurrence	ab?c matches “ac” and “abc”
\	Escapes special characters	\. matches literal dot
\w	Matches word characters	a-z, A-Z, 0-9 and underscore

2.3 Common Use Cases

1. Search Operations

- Advanced find/replace operations;
- Pattern matching in large text files;
- Content filtering.

2. Text Processing

- Finding patterns in text;
- Replacing specific text patterns;
- Extracting information;
- Parsing log files.

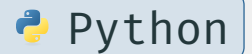
3. Data Validation

- Email addresses;
- Phone numbers;
- Postal codes;
- Passwords;
- URLs.

2.4 Advanced Concepts

2.4.1 Character Classes

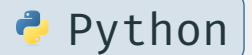
```
1 # Character class examples
2 pattern = r'[aeiou]' # Matches any vowel
3 pattern = r'[0-9]'   # Matches any digit
4 pattern = r'^0-9'    # Matches any non-digit
```



2.4 Advanced Concepts

2.4.2 Quantifiers and Groups

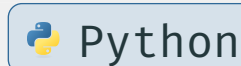
```
1 # Quantifiers
2 pattern = r'\d{3}'      # Exactly 3 digits
3 pattern = r'\d{2,4}'    # Between 2 and 4 digits
4 pattern = r'\d{2,}'     # 2 or more digits
5
6 # Groups
7 pattern = r'(\w+)\s+\1' # Matches repeated words
```



2.4 Advanced Concepts

2.4.3 Common Regex Functions in Python

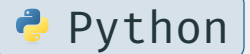
```
1  import re
2
3  text = "The price is $19.99"
4
5  # Different matching functions
6  re.search(r'\$\d+\.\d+', text)    # Finds first match
7  re.findall(r'\$\d+\.\d+', text)   # Finds all matches
8  re.sub(r'\$(\d+\.\d+)', r'\1', text) # Substitution
9
10 # Splitting text
11 re.split(r'\s+', text) # Split on whitespace
```



2.5 Python Implementation

2.5.1 Basic Pattern Matching

```
1  import re
2  # Simple pattern matching
3  text = "The quick brown fox jumps over the lazy dog"
4  pattern = r"fox"
5  # Search for pattern
6  match = re.search(pattern, text)
7  if match:
8      print(f"Found '{pattern}' at position: {match.start()}-
          {match.end()}")
9  # Find all occurrences
10 words = re.findall(r"\w+", text)
11 print(f"All words: {words}")
```



Python

2.5 Python Implementation

2.5.2 Email Validation Example

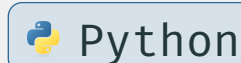
```
1  def is_valid_email(email):  
2      pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'  
3      return bool(re.match(pattern, email))  
4  # Test cases  
5  emails = [  
6      "user@example.com", # ✓  
7      "invalid.email@com", # ✗  
8      "user.name@bizerte.r-iset.tn", # ✓  
9      "@invalid.com" # ✗  
10 ]  
11 for email in emails:  
12     print(f"{email} → {'Valid' if is_valid_email(email) else  
        'Invalid'}")
```

 Python

2.5 Python Implementation

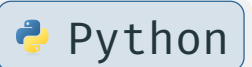
2.5.3 Phone Number Formatting

```
1  def format_phone_number(phone):
2      # Remove all non-digit characters
3      digits = re.sub(r'\D', '', phone)
4
5      # Format as (XXX) XXX-XXXX
6      if len(digits) == 10:
7          pattern = r'(\d{3})(\d{3})(\d{4})'
8          formatted = re.sub(pattern, r'(\1) \2-\3', digits)
9          return formatted
10
11     return "Invalid phone number"
```



2.5 Python Implementation

```
1  # Test cases
2  numbers = [
3      "1234567890",
4      "123-456-7890",
5      "(123) 456-7890",
6      "12345"
7  ]
8
9  for number in numbers:
10     print(f"{number} → {format_phone_number(number)}")
```



2.6 Character Shorthands & Non-Capturing Groups

Built-in shorthands match common character classes

`\s` — any whitespace character (space, tab, newline...)

`\s+`

Matches the gap in "hello world"

`\S` — any NON-whitespace character

`\S+`

Matches "hello" and "world" separately

2.6 Character Shorthands & Non-Capturing Groups

`\w` `\W` — any NON-word character (opposite of `\w`)

`\W+`

Matches " !@#" in "hello!@#world"

`\b` `\B` — word boundary (zero-width, between `\w` and `\W`)

`\bcat\b`

Matches "cat" in "the cat sat" — not in "concatenate"

`(?: ... )` — non-capturing group: group without storing a backreference

`(?:ha)+`

Matches "hahaha" as one unit — no capture group created

2.6 Character Shorthands & Non-Capturing Groups

Token	Name	Matches
\s	Whitespace	space · tab · newline · form-feed · carriage-return
\S	Non-whitespace	anything \s does not match
\W	Non-word char	anything \w does not match — punctuation, spaces, symbols
\b	Word boundary	zero-width position between word and non-word chars
(?:)	Non-capturing group	groups a sub-pattern without a capture slot

2.7 Lookahead and lookbehind

2.7.1 Lookahead — Positive & Negative

Lookahead checks what **follows** the current position — without consuming any characters (zero-width assertion).

POSITIVE `(?=...)` — match only if followed by

```
\d+(?= dollars)
```

Matches 100 in "100 dollars" — not in "100 euros"

NEGATIVE `(?!...)` — match only if NOT followed by

```
\d+(?! dollars)
```

Matches 100 in "100 euros" — not in "100 dollars"

2.7 Lookahead and lookbehind

2.7.2 Lookbehind — Positive & Negative

Lookbehind checks what **precedes** the current position — also zero-width. Most engines require fixed-length lookbehind patterns.

POSITIVE `(?<=...)` — match only if preceded by

`(?<=\$)\d+`

Matches 500 in "\$500" — not in "€500"

NEGATIVE `(?<!...)` — match only if NOT preceded by

`(?<!\$)\d+`

Matches 500 in "€500" — not in "\$500"

2.7 Lookahead and lookbehind

Type	Syntax	Meaning
Positive Lookahead	(?= ...)	followed by
Negative Lookahead	(?! ...)	NOT followed by
Positive Lookbehind	(?<= ...)	preceded by
Negative Lookbehind	(?<!= ...)	NOT preceded by

2.8 Best Practices

1. Use Raw Strings

- Always prefix regex patterns with `r` to avoid escape character issues

```
1 pattern = r'\d+' # Better than '\d+'
```

 Python

2. Compile Frequently Used Patterns

```
1 email_pattern = re.compile(r'^[\w\.-]+@[\w\.-]+\.\w+$')
2 # Use multiple times
3 email_pattern.match(email1)
4 email_pattern.match(email2)
```

 Python

2.8 Best Practices

3. Be Specific

- Make patterns as specific as possible to avoid false matches;
- Use start (^) and end (\$) anchors when matching whole strings.

4. Test Thoroughly

- Test with both valid and invalid inputs
- Include edge cases in your tests

```
1 def test_pattern(pattern, test_cases):  
2     regex = re.compile(pattern)  
3     for test, expected in test_cases:  
4         result = bool(regex.match(test))  
5         print(f"'{test}': {'v' if result == expected else 'x'}")
```



2.9 Common Pitfalls



Quote

*Some people, when confronted with a problem, think “I know, I’ll use regular expressions.”
Now they have two problems. - Jamie Zawinski*

2.9 Common Pitfalls

1. Greedy vs. Non-Greedy Matching

```
1 # Greedy (default)
2 re.findall(r'<.*>', '<tag>text</tag>') # ['<tag>text</tag>']
3
4 # Non-greedy: Add (lazy) `?`
5 re.findall(r'<.*?>', '<tag>text</tag>') # ['<tag>', '</tag>']
```



2. Performance Considerations

- Avoid excessive backtracking (*recursion*);
- Be careful with nested quantifiers;
- Use more specific patterns when possible.

2.10 Applications

1. Basic Pattern Matching

```
1 # Write a pattern to match dates in format DD/MM/YYYY
2 date_pattern = r'\d{2}/\d{2}/\d{4}'
```

 Python

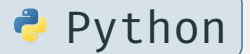
2. Data Extraction

```
1 # Extract all email addresses from text
2 text = "Contact us at support@example.com or sales@example.com"
3 emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', text)
```

 Python

3. Password Validation

```
1 def is_strong_password(password):  
2     # At least 8 chars, 1 upper, 1 lower, 1 digit, 1 special  
3     pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$' # Positive Lookahead  
4     return bool(re.match(pattern, password))
```



2.10 Applications

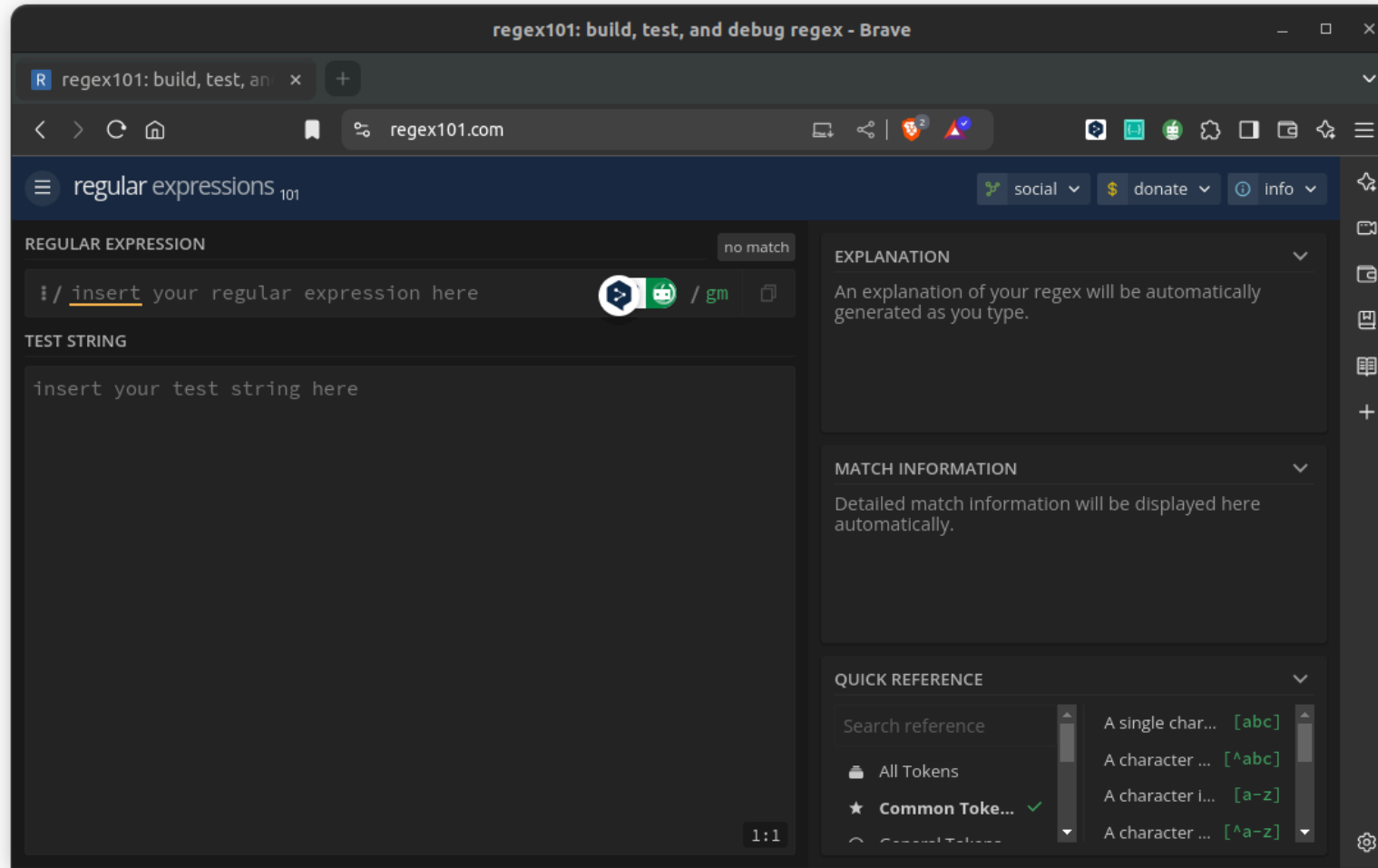


Figure 1: Build, test and debug regex patterns.

2.10 Applications

Task 1:

Write a regex pattern to match all occurrences of the word “python” (case-insensitive) in a string.

```
pattern = re.compile(r“python”, re.IGNORECASE)
```

2.10 Applications

Task 2:

Write a regex pattern to validate email addresses.

```
pattern = re.compile(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$")
```

2.10 Applications

Task 3:

Extract all phone numbers from a text where numbers can be in format XXX-XXX-XXXX or (XXX) XXX-XXXX.

```
pattern = re.compile(r"(\d{3}-\d{3}-\d{4})|(\d{3}\)\s\d{3}-\d{4})")
```


2.10 Applications

Task 4:

Extract the username and domain from email addresses.

```
pattern = re.compile(r"^([a-zA-Z0-9._%+-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$")
```

2.10 Applications

Task 5:

Match whole words “code” and “coding” but not words that contain them like “encoder” or “decode”.

```
pattern = re.compile(r"\b(code|coding)\b")
```

3. Text Tokenization

3.1 Introduction to Tokenization

Tokenization is the process of breaking down text into smaller units called tokens. These tokens can be words, characters, subwords, or phrases depending on the specific requirements of the **NLP** task.

3.2 Basic Regex-based Tokenization

3.2.1 Simple Word Tokenization

```
1  import re
2
3  def simple_word_tokenize(text):
4      # Split on whitespace and punctuation
5      tokens = re.findall(r'\b\w+\b', text)
6      return tokens
7
8  text = "Hello, world! This is a simple example."
9  tokens = simple_word_tokenize(text)
10 print(tokens)
```



```
['Hello', 'world', 'This', 'is', 'a', 'simple', 'example']
```

3.2 Basic Regex-based Tokenization

3.2.2 Advanced Regex Tokenization

```
1 def advanced_tokenize(text):
2     pattern = r"""
3         [a-zA-Z]+ | (?≤\$)\d+(?:\.\d+)? | \d+(?:\.\d+)?(?:!|=)
4     """
5     tokens = re.findall(pattern, text)
6     return tokens
7
8 text = "The price is $19.99, and the discount is 15%!"
9 print(advanced_tokenize(text))
```

 Python

```
['The', 'price', 'is', '19.99', 'and', 'the', 'discount', 'is', '15']
```

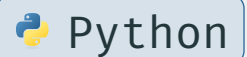
3.3 NLTK Tokenization

NLTK (*Natural Language Toolkit*) is a Python library offering tools for tokenization, stemming, part-of-speech tagging, and more, making it ideal for text analysis and linguistic research.

While it's widely used for education and prototyping, **NLTK** is less efficient for large-scale applications compared to modern libraries like **spaCy** or **Hugging Face Transformers**.

3.3.1 Installation and Usage

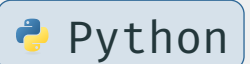
```
1 import nltk
2 nltk.download('punkt_tab') # Required for word and sentence
   tokenization
```



3.3 NLTK Tokenization

3.3.2 Word Tokenization

```
1 from nltk.tokenize import word_tokenize,  
   TreebankWordTokenizer  
2 text = "Don't hesitate to use NLTK's tokenizer."  
3 tokens = word_tokenize(text)  
4 print(tokens)  
5 # Output: ['Do', "n't", 'hesitate', 'to', 'use', 'NLTK', "'s",  
   'tokenizer', '.']  
6 treebank = TreebankWordTokenizer()  
7 tokens = treebank.tokenize(text)  
8 print(tokens)
```

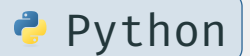


```
['Do', "n't", 'hesitate', 'to', 'use', 'NLTK', "'s", 'tokenizer', '.']
```


3.3 NLTK Tokenization

3.3.3 Sentence Tokenization

```
1 from nltk.tokenize import sent_tokenize
2 text = """Mr. Smith bought a car. He loves driving it!
3       What will he buy next? Only time will tell."""
4 sentences = sent_tokenize(text)
5 print(sentences)
```



```
['Mr. Smith bought a car.', 'He loves driving it!', 'What will he buy next?', 'Only time will tell.']
```

3.3 NLTK Tokenization

3.3.4 Regular Expression Tokenizer

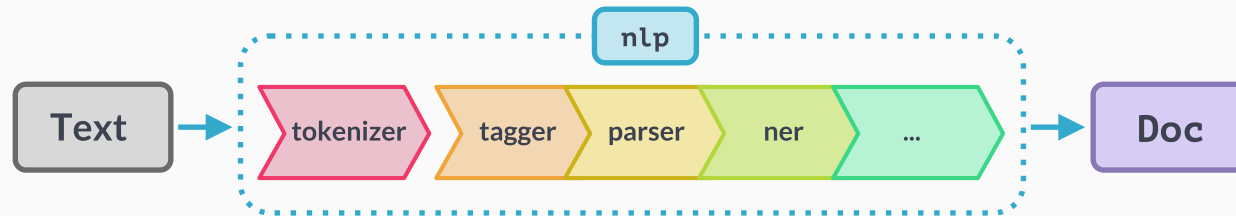
```
1 from nltk.tokenize import RegexpTokenizer
2 tokenizer = RegexpTokenizer(pattern=r'\w+|[^ \w\s]+' )
3 text = "Hello, World! How's it going?"
4 tokens = tokenizer.tokenize(text)
5 print(tokens)
```

 Python

```
['Hello', ',', 'World', '!', 'How', "'", 's', 'it', 'going', '?']
```

3.4 spaCy Tokenization

spaCy¹² offers faster, more efficient tokenization with built-in linguistic annotations, making it better suited for production pipelines than rule-based alternatives.



3.4.1 Installation and Usage

```
1 python -m spacy download en_core_web_sm
```



```
1 import spacy
```

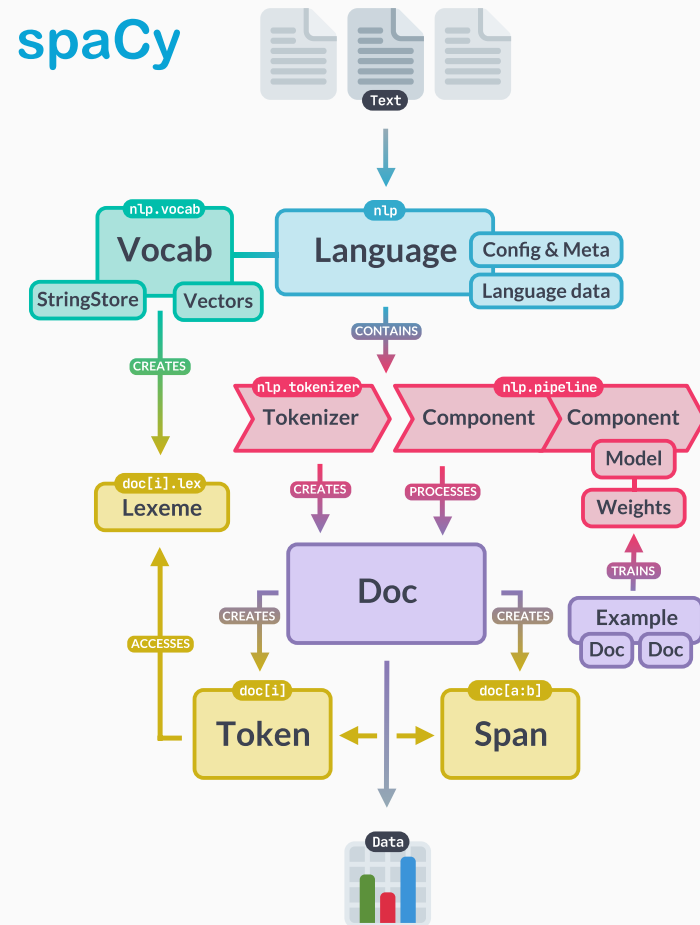
```
2 nlp = spacy.load('en_core_web_sm') # Load the English model
```



¹<https://spacy.io>

²Free online course **Advanced NLP with spaCy** is available at <https://course.spacy.io>

3.4 spaCy Tokenization



3.4 spaCy Tokenization

```
1 def spacy_tokenize(text):
2     doc = nlp(text)
3     return [token.text for token in doc]
4
5 text = "spaCy's tokenizer is industrial-strength!"
6 tokens = spacy_tokenize(text)
7 print(tokens)
```

 Python

```
["spaCy", "s", "tokenizer", "is", "industrial", "-", "strength", "!"]
```

3.4 spaCy Tokenization

3.4.2 Tokenization with Linguistic Features

```
1  text = "Apple's stock price rose 5.2% to $200 in 2024. Wow!"
2
3  # Tokenize and analyze the text
4  doc = nlp(text) # Transform the text into a Doc object
5  # Extract tokens with linguistic features
6  for token in doc:
7      print(
8          f"Token: {token.text:<10}",
9          f"POS: {token.pos_:<8}",
10         f"Lemma: {token.lemma_:<10}",
11         f"Is punctuation? {token.is_punct}"
12     )
13
```



3.4 spaCy Tokenization

14	" " "			
15	Token: Apple False	POS: PROPN	Lemma: Apple	Is punctuation?
16	Token: 's False	POS: PART	Lemma: 's	Is punctuation?
17	Token: stock False	POS: NOUN	Lemma: stock	Is punctuation?
18	Token: price False	POS: NOUN	Lemma: price	Is punctuation?
19	Token: rose False	POS: VERB	Lemma: rise	Is punctuation?
20	Token: 5.2 False	POS: NUM	Lemma: 5.2	Is punctuation?
21	Token: %	POS: NOUN	Lemma: %	Is punctuation? True

3.4 spaCy Tokenization

22	Token: to False	POS: ADP	Lemma: to	Is punctuation?
23	Token: \$ False	POS: SYM	Lemma: \$	Is punctuation?
24	Token: 200 False	POS: NUM	Lemma: 200	Is punctuation?
25	...			
26	" " "			

3.4 spaCy Tokenization

```
1  def analyze_tokens(text):
2      """
3      Analyze the tokens in a text using spaCy.
4      """
5      doc = nlp(text)
6      for token in doc:
7          print(f"""
8              Text: {token.text}
9              Lemma: {token.lemma_}
10             POS: {token.pos_}
11             Is stop word: {token.is_stop}
12             """)
13
14  text = "Running quickly through the forest"
```



Python

3.4 spaCy Tokenization

```
15 analyze_tokens(text)
16 """
17 Text: Running
18 Lemma: run
19 POS: VERB
20 Is stop word: False
21
22 Text: quickly
23 Lemma: quickly
24 POS: ADV
25 Is stop word: False
26 """
```

3.5 Polyglot Tokenization

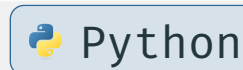
Polyglot is an open-source **NLP** library designed for multilingual text processing, supporting over 130 languages for tasks like tokenization, named entity recognition, and part-of-speech tagging. Unlike **spaCy** or **NLTK**, it specializes in low-resource languages, making it useful for linguistic diversity but less optimized for speed or deep learning integration.

3.5.1 Installation and Usage

```
1 pip install polyglot
```



```
1 from polyglot.text import Text
```



3.5 Polyglot Tokenization

3.5.2 Basic Usage

```
1  def polyglot_tokenize(text, lang=None):  
2      text = Text(text, hint_language_code=lang)  
3      return text.words  
4  
5  arabic_text = "! مرحبا أيها العالم"  
6  english_text = "Hello, world!"  
7  spanish_text = "¡Hola, mundo!"  
8  samples = {"ar": arabic_text, "en": english_text, "es": spanish_text}  
9  
10 for k, v in samples:  
11     tokens = polyglot_tokenize(v, k)  
12     print(f"Original: {sample}")  
13     print(f"Tokens: {tokens}\n")
```



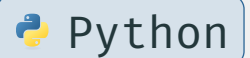
Python

3.5 Polyglot Tokenization

```
14 """
15 Original: ! مرحبا أيها العالم
16 Tokens: ['!', 'العالم', 'أيها', 'مرحبا']
17
18 Original: Hello, world!
19 Tokens: ['Hello', ',', 'world', '!']
20
21 Original: ¡Hola, mundo!
22 Tokens: ['¡', 'Hola', ',', 'mundo', '!']
23 """
```

3.6 Regex vs. NLTK vs. spaCy vs. Polyglot

```
1  text = "Don't forget those in need!"
2
3  print("Regex:", simple_word_tokenize(text))
4  # Regex: ['Don', 't', 'forget', 'those', 'in', 'need']
5  print("NLTK:", word_tokenize(text))
6  # NLTK: ['Do', "n't", 'forget', 'those', 'in', 'need', '!']
7  print("spaCy:", spacy_tokenize(text))
8  # spaCy: ['Do', "n't", 'forget', 'those', 'in', 'need', '!']
9  print("Polyglot:", polyglot_tokenize(text))
10 # Polyglot: ["Don't", 'forget', 'those', 'in', 'need', '!']
```



Python

3.6 Regex vs. NLTK vs. spaCy vs. Polyglot

1. **Regex-based Tokenization**

-  Simple, custom tokenization rules
-  Fast, flexible, easy to modify
-  Can't handle complex linguistic cases

2. **NLTK**

-  Academic and research projects
-  Rich features, well-documented
-  Slower than some alternatives

3. **spaCy**

-  Production environments
-  Fast, modern, good defaults
-  Larger memory footprint

4. **Polyglot**

-  Multilingual projects
-  Excellent language coverage
-  Can be slower, fewer features

3.7 Best Practices

1. Choose the Right Tokenizer

```
1 def select_tokenizer(text, language='en',  
2   needs_speed=False):  
3     return spacy_tokenize(text)  
4   elif language != 'en':  
5     return polyglot_tokenize(text)  
6   else:  
7     return word_tokenize(text)
```



3.7 Best Practices

```
1  # Sample texts for testing different tokenization paths
2  test_texts = {
3      "english_basic": """This sentence contains simple punctuation,
4      numbers (123), and some special characters !@#$."""
5      "english_complex": """Mr. Smith's car broke down at 3:30 P.M.
6      "This is terrible," he thought."""
7      "multilingual": """Hello in French is Bonjour. In Spanish, hello
8      is ¡Hola!"""
9      "technical": """Python3 supports utf-8 encoding. Variables use
10     snake_case by convention."""
11 }
```

 Python

3.7 Best Practices

Test case 1: Basic English, no speed requirement

```
1 print("Test 1: Basic English (default settings)")
2 result1 = select_tokenizer(test_texts["english_basic"])
3 print(f"Tokens: {result1}\n")
```

 Python

```
['This', 'sentence', 'contains', 'simple', 'punctuation', ',', 'numbers', '(', '123', ')', ',', 'and', 'some', 'special', 'characters', '!', '@', '#', '$', ':']
```

3.7 Best Practices

Test case 2: Non-English text

```
1 print("Test 2: Multilingual text")
2 result2 = select_tokenizer(test_texts["multilingual"], language="fr")
3 print(f"Tokens: {result2}\n")
```

 Python

```
['Hello', 'in', 'French', 'is', 'Bonjour', '.', 'In', 'Spanish', ',', 'hello', 'is', '!', 'Hola', '!']
```

3.7 Best Practices

Test case 3: English with speed requirement

```
1 print("Test 3: English with speed optimization")
2 result3 = select_tokenizer(test_texts["english_complex"],
3                             needs_speed=True)
3 print(f"Tokens: {result3}\n")
```

 Python

```
['Mr.', 'Smith', "'s", 'car', 'broke', 'down', 'at', '3:30', 'P.M.', "'", 'This', 'is', 'terrible', ',', "'", 'he', 'thought', '.']
```

3.7 Best Practices

Test case 4: Technical text with special characters

```
1 print("Test 4: Technical text")
2 result4 = select_tokenizer(test_texts["technical"])
3 print(f"Tokens: {result4}")
```

 Python

```
['Python3', 'supports', 'utf-8', 'encoding', '.', 'Variables', 'use', 'snake_case', 'by', 'convention', '.']
```

3.7 Best Practices

2. Pre-processing

```
1 def preprocess_text(text):  
2     # Convert to lowercase  
3     text = text.lower()  
4     # Remove extra whitespace  
5     text = re.sub(r'\s+', ' ', text).strip()  
6     # Remove special characters (if needed)  
7     text = re.sub(r'^\w\s]', '', text)  
8     return text
```



‘ Hello WORLD ‘

‘!@#\$%^&*()_+’

‘Hello\n\tWorld\n Python’

‘hello world’

‘ _ ’

‘hello world python’

3.7 Best Practices

3. Handling Special Cases

```
1 def handle_special_cases(tokens):  
2     """Expand contractions in a token list."""  
3     contractions = {"n't": "not", "'s": "is", "'re": "are"} # Added  
    common cases  
4     expanded_tokens = []  
5     for token in tokens:  
6         expanded_tokens.append(contractions.get(token, token)) #  
            dict.get(key, default)  
7     return expanded_tokens
```

 Python

```
['They', 'ca', "n't", 'believe', 'it', "'s", 'already', 'Friday']
```

```
['They', 'ca', 'not', 'believe', 'it', 'is', 'already', 'Friday']
```


3.8 Named Entity Recognition (NER)

3.8.1 What is Named Entity Recognition?

Named Entity Recognition (NER) is a natural language processing technique that identifies and classifies named entities (key elements) in text into predefined categories.

The categories include but are not limited to:

- Person names (e.g., “Barack Obama”, “Shakespeare”)
- Organizations (e.g., “Microsoft”, “United Nations”)
- Locations (e.g., “Paris”, “Mount Everest”)
- Date/Time expressions (e.g., “June 2024”, “last Monday”)
- Monetary values (e.g., “\$1000”, “€50”)
- Percentages (e.g., “25%”, “three-quarters”)

3.8 Named Entity Recognition (NER)

3.8.2 Usage Example

Input text: “Apple CEO Tim Cook announced new iPhone models in California last September.”

Identified entities:

- Apple (ORG)
- Tim Cook (PERSON)
- iPhone (ORG)
- California (GPE)
- September (DATE)

Apple ORG CEO Tim Cook PERSON announced new iPhone ORG models in California GPE last September DATE

3.8 Named Entity Recognition (NER)

```
1 import spacy
2 from spacy import displacy
3
4 nlp = spacy.load('en_core_web_sm')
5
6 doc = nlp("Apple CEO Tim Cook announced new iPhone models in California
7 last September.")
8 displacy.serve(doc, style='ent')
```



3.8 Named Entity Recognition (NER)



Extracting Named Entities

[Codes/extracting_named_entity.py](#)

3.8 Named Entity Recognition (NER)

3.8.3 Common Use Cases

1. Information Extraction

- Extracting company names from news articles
- Identifying people mentioned in social media posts
- Finding locations in travel blogs

2. Document Classification

- Categorizing documents based on mentioned organizations
- Sorting news articles by location
- Grouping documents by date mentions

3.8 Named Entity Recognition (NER)

3. Relationship Extraction

- Identifying business relationships between companies
- Finding connections between people
- Mapping event locations and dates

4. Content Enrichment

- Adding metadata to documents
- Linking entities to knowledge bases
- Creating document summaries

4. Text Processing and Visualization

4.1 Fundamental Concepts and Definitions

4.1.1 What is Text Preprocessing?

Text preprocessing is the process of cleaning and transforming raw text into a format that's more suitable for analysis.

Think of it as preparing ingredients before cooking - just as you wash and chop vegetables before cooking, you clean and standardize text before analysis.

1. Raw text: *"RT @username: Check out our new product!!! It's AMAZING... 😊 www.example.com #awesome"*
2. Preprocessed text: *"check out our new product it is amazing"*

4.1 Fundamental Concepts and Definitions

4.1.2 Key Preprocessing Steps

4.1.2.1 Tokenization

The process of breaking down text into individual units (tokens), typically words or subwords.

1. Sentence: *"I love natural language processing!"*
2. Tokens: [*"I", "love", "natural", "language", "processing", "!"*]
1. Sentence: *"Bizerte City is beautiful"*
2. Tokens: [*"Bizerte", "City", "is", "beautiful"*]

4.1 Fundamental Concepts and Definitions

4.1.2.2 Stop Word Removal

Stop word removal is a **text preprocessing step** that filters out high-frequency words carrying little semantic value before analysis.

Common stop words in English include:

- a, an, the
- is, are, was
- in, on, at, by

Eliminating common words that typically don't carry significant meaning. Not all words contribute equally to meaning.

4.1 Fundamental Concepts and Definitions

Why do we remove stop words?

- ✓ **Noise reduction** — less clutter in data
- ✓ **Dimensionality reduction** — smaller vocabulary
- ✓ **Better model performance** — stronger signal
- ✓ **Faster computation** — less data to process

Before:

“The cat is sitting on the mat”

After:

“cat sitting mat”

4.1 Fundamental Concepts and Definitions

When should we keep stop words?

Stop word removal is **not always appropriate**. Avoid it when:

Task	Reason to Keep
Sentiment Analysis	“not good” $\neq$ “good” — negations matter
Machine Translation	Grammar depends on function words
Language Modeling	Fluency requires grammatical structure
Named Entity Recognition	Context words aid entity detection

4.1 Fundamental Concepts and Definitions

1. Stop words are high-frequency, low-information words.
2. Removing them reduces noise and improves efficiency.
3. Always consider the task — removal can **hurt** some models.
4. Use library defaults or build domain-specific lists.
5. It's one step in a broader preprocessing pipeline.

4.1 Fundamental Concepts and Definitions

4.1.2.3 Lemmatization

Reducing words to their base or dictionary form (lemma).

1 am, are, is → be

2 running, ran, runs → run

3 better, best → good

4 wolves → wolf

4.1 Fundamental Concepts and Definitions

4.1.2.4 Stemming

Reducing words to their root form by removing affixes, often resulting in non-dictionary words.

1 running → run

2 fishing → fish

3 completely → complet

4 authentication → authent

4.1 Fundamental Concepts and Definitions

Let's analyze a customer review:

Original review:

"I've been using this phone for 3 months now... It's AMAZING!!! The battery life is incredible, and the camera takes beautiful pics. Can't believe how good it is :) Would definitely recommend to my friends & family!!!"

4.1 Fundamental Concepts and Definitions

1. **Cleaning:**

“i have been using this phone for three months now it is amazing the battery life is incredible and the camera takes beautiful pictures cannot believe how good it is would definitely recommend to my friends and family”

2. **Tokenization:**

["i", "have", "been", "using", "this", "phone", "for", "three", "months", ...]

3. **Stop Word Removal:**

["phone", "three", "months", "amazing", "battery", "life", "incredible", "camera", "takes", "beautiful", "pictures", "good", "definitely", "recommend", "friends", "family"]

4. **Lemmatization:**

["phone", "three", "month", "amazing", "battery", "life", "incredible", "camera", "take", "beautiful", "picture", "good", "definitely", "recommend", "friend", "family"]

4.1 Fundamental Concepts and Definitions

4.1.3 Bag of Words (BoW)

A text representation method that describes the occurrence of words within a document. It creates a vocabulary of unique words and represents each document as a vector of word frequencies.

Documents:

The cat likes milk
The dog hates milk

Vocabulary:

["the", "cat", "dog", "likes",
"hates", "milk"]

BoW representations:

- Doc 1: [1, 1, 0, 1, 0, 1]
- Doc 2: [1, 0, 1, 0, 1, 1]

4.1 Fundamental Concepts and Definitions

I love machine learning
I love deep learning
Deep learning is a subset of machine learning

4.1.3.1 Vocabulary

The unique words across all documents: “I”, “love”, “machine”, “learning”, “deep”, “is”, “a”, “subset”, “of”

4.1.3.2 Document Vectors

Document 1: [1, 1, 1, 1, 0, 0, 0, 0, 0]

Document 2: [1, 1, 0, 1, 1, 0, 0, 0, 0]

Document 3: [0, 0, 1, 2, 1, 1, 1, 1, 1]

4.1 Fundamental Concepts and Definitions

4.1.3.3 BoW Matrix

Document	I	love	machine	learning	deep	is	a	subset	of
Doc 1	1	1	1	1	0	0	0	0	0
Doc 2	1	1	0	1	1	0	0	0	0
Doc 3	0	0	1	2	1	1	1	1	1

4.1.3.4 Document Similarity Analysis

Let's verify document similarity by calculating dot products between document vectors:

$$\text{Doc 1} \cdot \text{Doc 2} = (1 \times 1) + (1 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 1) + (0 \times 0) + (0 \times 0) + (0 \times 0) + (0 \times 0) = 3$$

$$\text{Doc 1} \cdot \text{Doc 3} = (1 \times 0) + (1 \times 0) + (1 \times 1) + (1 \times 2) + (0 \times 1) + (0 \times 1) + (0 \times 1) + (0 \times 0) + (0 \times 1) = 3$$

$$\text{Doc 2} \cdot \text{Doc 3} = (1 \times 0) + (1 \times 0) + (0 \times 1) + (1 \times 2) + (1 \times 1) + (0 \times 1) + (0 \times 1) + (0 \times 1) + (0 \times 1) = 3$$

4.1 Fundamental Concepts and Definitions

For proper similarity comparison, we should normalize using cosine similarity:

$$\cos(\theta) = \frac{(A) \cdot (B)}{\|(A)\| \times \|(B)\|}$$

Doc 1 magnitude: $\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{4} = 2$

Doc 2 magnitude: $\sqrt{1^2 + 1^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{4} = 2$

Doc 3 magnitude: $\sqrt{0^2 + 0^2 + 1^2 + 2^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2} = \sqrt{10} \approx 3.16$

Cosine similarities:

Doc 1 & Doc 2	Doc 1 & Doc 3	Doc 2 & Doc 3
$\frac{3}{2 \times 2} = 0.75$	$\frac{3}{2 \times 3.16} \approx 0.47$	$\frac{3}{2 \times 3.16} \approx 0.47$

Doc 1 and **Doc 2** are more similar to each other (0.75) than either is to **Doc 3** (0.47).

4.1 Fundamental Concepts and Definitions

4.1.4 Term Frequency-Inverse Document Frequency (TF-IDF)

A numerical statistic that reflects how important a word is to a document in a collection of documents.

Consider these news articles:

The new iPhone features advanced AI capabilities

The new Android phone launches today

The weather is nice today

The word “the” appears in all documents, so it gets a low IDF score.

The word “iPhone” appears in only one document, so it gets a high IDF score.

4.1 Fundamental Concepts and Definitions

4.1.4.1 TF-IDF Calculation

It is calculated by multiplying two metrics: Term Frequency (TF) and Inverse Document Frequency (IDF).

The TF-IDF score for a term t in document d from a document collection D is calculated as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

4.1 Fundamental Concepts and Definitions

1. **Term Frequency (TF)** measures how frequently term t appears in document d :

$$\text{TF}(t, d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of terms in } d}$$

2. **Inverse Document Frequency (IDF)** measures the importance of term t across all documents:

$$\text{IDF}(t, D) = \log\left(\frac{N}{\text{DF}(t)}\right)$$

Where:

- N is the total number of documents in collection D
- $\text{DF}(t)$ is the number of documents containing term t

4.1 Fundamental Concepts and Definitions

4.1.4.2 Computing Example

- **Document 1:** “This smartphone has a great camera and long battery life”
- **Document 2:** “The camera on this laptop is decent but the keyboard is excellent”
- **Document 3:** “Battery life is crucial for any smartphone”
- **Document 4:** “This laptop has a fast processor and excellent keyboard”

Calculate TF for each term in each document: First, let’s count terms in each document (after removing common words like “this”, “has”, “a”, “is”, “the”, etc.):

4.1 Fundamental Concepts and Definitions

For **Document 1** (6 terms):

- $\text{TF}(\text{smartphone}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{great}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{camera}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{long}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{battery}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{life}) = \frac{1}{6} \approx 0.167$

For **Document 2** (5 terms):

- $\text{TF}(\text{camera}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{laptop}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{decent}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{keyboard}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{excellent}) = \frac{1}{5} = 0.2$

4.1 Fundamental Concepts and Definitions

For **Document 3** (4 terms):

- $\text{TF}(\text{battery}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{life}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{crucial}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{smartphone}) = \frac{1}{4} = 0.25$

For **Document 4** (5 terms):

- $\text{TF}(\text{laptop}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{fast}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{processor}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{excellent}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{keyboard}) = \frac{1}{5} = 0.2$

4.1 Fundamental Concepts and Definitions

Calculate IDF for each term across all documents:

Total documents (N) = 4

- $\text{IDF}(\text{smartphone}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{great}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{camera}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{long}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{battery}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{life}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{laptop}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{decent}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{keyboard}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{excellent}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{crucial}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{fast}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{processor}) = \log\left(\frac{4}{1}\right) \approx 0.602$

4.1 Fundamental Concepts and Definitions

Calculate TF-IDF for each term in each document:

For **Document 1**:

- $\text{TF-IDF}(\text{smartphone}, d_1) \approx 0.05$
- $\text{TF-IDF}(\text{great}, d_1) \approx 0.1$
- $\text{TF-IDF}(\text{camera}, d_1) \approx 0.05$
- $\text{TF-IDF}(\text{long}, d_1) \approx 0.1$
- $\text{TF-IDF}(\text{battery}, d_1) \approx 0.05$
- $\text{TF-IDF}(\text{life}, d_1) \approx 0.05$

For **Document 2**:

- $\text{TF-IDF}(\text{camera}, d_2) \approx 0.06$
- $\text{TF-IDF}(\text{laptop}, d_2) \approx 0.06$
- $\text{TF-IDF}(\text{decent}, d_2) \approx 0.12$
- $\text{TF-IDF}(\text{keyboard}, d_2) \approx 0.06$
- $\text{TF-IDF}(\text{excellent}, d_2) \approx 0.06$

4.1 Fundamental Concepts and Definitions

For **Document 3**:

- $\text{TF-IDF}(\text{battery}, d_3) \approx 0.1$
- $\text{TF-IDF}(\text{life}, d_3) \approx 0.1$
- $\text{TF-IDF}(\text{crucial}, d_3) \approx 0.2$
- $\text{TF-IDF}(\text{smartphone}, d_3) \approx 0.1$

For **Document 4**:

- $\text{TF-IDF}(\text{laptop}, d_4) \approx 0.06$
- $\text{TF-IDF}(\text{fast}, d_4) \approx 0.12$
- $\text{TF-IDF}(\text{processor}, d_4) \approx 0.12$
- $\text{TF-IDF}(\text{excellent}, d_4) \approx 0.06$
- $\text{TF-IDF}(\text{keyboard}, d_4) \approx 0.06$

4.1 Fundamental Concepts and Definitions

The terms with the highest TF-IDF scores in each document:

<i>Document</i>	<i>Highest TF-IDF Terms</i>
Document 1	“great” (0.1) and “long” (0.1)
Document 2	“decent” (0.12)
Document 3	“crucial” (0.2)
Document 4	“fast” (0.12) and “processor” (0.12)

This demonstrates how TF-IDF identifies the most distinctive terms in each document. Terms that appear in only one document (like “crucial”, “processor”, or “fast”) receive higher scores than terms that appear across multiple documents (like “smartphone” or “camera”).

4.2 Understanding N-grams

An n-gram is a contiguous sequence of n items extracted from a text or speech sample. These items can be characters, words, syllables, or other linguistic units depending on the application context.

Unigrams (1-grams) Individual items (*e.g., single words like “computer”*)

Bigrams (2-grams) Pairs of consecutive items (*e.g., “natural language”*)

Trigrams (3-grams) Three consecutive items (*e.g., “machine learning algorithm”*)

4-grams, 5-grams etc. Longer sequences following the same pattern

N-gram models operate on the **Markov** assumption that the probability of a word depends primarily on the previous $n - 1$ words, creating a probabilistic language model. While larger n values capture more context, they also increase computational complexity and data sparsity challenges.

4.2 Understanding N-grams

Chain Rule & n-gram (Markov) Approximation

The general **chain rule** of probability decomposes a joint distribution over a sequence of words:

$$P(w_1 w_2 \dots w_n) = P(w_1) \prod_{i=2}^n P(w_i \mid w_1 w_2 \dots w_{i-1})$$

The **n-gram approximation** (nth-order Markov assumption) simplifies each conditional to depend only on the $n - 1$ immediately preceding words:

$$P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-1} \dots w_{i-n+1})$$

4.2 Understanding N-grams

N-grams serve as fundamental building blocks in various **NLP** applications:

- Text prediction and auto-completion systems
- Machine translation
- Speech recognition
- Information retrieval and search engines
- Text classification and sentiment analysis
- Spelling correction
- Computational biology (for analyzing **DNA** sequences)

4.2 Understanding N-grams

Task 6:

1. Identify all unigrams and their frequencies
2. List all possible bigrams
3. List all possible trigrams
4. Calculate the probability of “sat” following “cat”
5. If we add the sentence “The cat ate the fish,” calculate the probability of “the” being followed by “mat”

The cat sat on the mat.

4.2 Understanding N-grams

1. Unigrams and frequencies

- “The”: 1
- “cat”: 1
- “sat”: 1
- “on”: 1
- “the”: 1
- “mat”: 1

2. All possible bigrams:

- “The cat”
- “cat sat”
- “sat on”
- “on the”
- “the mat”

3. All possible trigrams:

- “The cat sat”
- “cat sat on”
- “sat on the”
- “on the mat”

If treating “The” and “the” as case-insensitive, then “the” would have a frequency of 2.

4.2 Understanding N-grams

4. Probability of “sat” following “cat”:

$$P(\text{sat}|\text{cat}) = \frac{\text{Count}(\text{cat sat})}{\text{Count}(\text{cat})} = \frac{1}{1} = 1.0$$

5. With added sentence “The cat ate the fish”:

Combined text: “The cat sat on the mat. The cat ate the fish.”

Updated frequencies for “the”: 2 (if case-insensitive, counting “The” and “the”) Count of “the mat”: 1

$$P(\text{mat}|\text{the}) = \frac{\text{Count}(\text{the mat})}{\text{Count}(\text{the})} = \frac{1}{2} = 0.5$$

4.2 Understanding N-grams

Task 7:

1. Count all unigrams, bigrams, and trigrams in the text (treat the text as case-insensitive)
2. Calculate the following probabilities:
 - $P(\text{ran}|\text{dog})$: Probability that “ran” follows “dog”
 - $P(\text{cat}|\text{The})$: Probability that “cat” follows “The”
 - $P(\text{high}|\text{jumped})$: Probability that “high” follows “jumped”
 - Using the bigram model, identify the most likely word to follow “The dog”
 - Using the bigram model, calculate the probability of the sequence “The cat ran”

The dog ran fast. The dog jumped high. A cat ran past the dog. The cat was quick.

4.2 Understanding N-grams

Q1 — N-gram Counts (treating text as case-insensitive)

Unigrams (1-grams):	Bigrams (2-grams):	Trigrams (3-grams):
• “the”: 4 • “dog”: 3 • “ran”: 2 • “fast”: 1 • “jumped”: 1 • “high”: 1 • “a”: 1 • “cat”: 2 • “past”: 1 • “was”: 1 • “quick”: 1	• “the dog”: 3 • “dog ran”: 1 • “ran fast”: 1 • “fast the”: 1 • “dog jumped”: 1 • “jumped high”: 1 • “high a”: 1 • “a cat”: 1 • “cat ran”: 1 • “ran past”: 1 • “past the”: 1 • “dog the”: 1 • “the cat”: 1 • “cat was”: 1 • “was quick”: 1	• “the dog ran”: 1 • “dog ran fast”: 1 • “ran fast the”: 1 • “fast the dog”: 1 • “the dog jumped”: 1 • “dog jumped high”: 1 • “jumped high a”: 1 • “high a cat”: 1 • “a cat ran”: 1 • “cat ran past”: 1 • “ran past the”: 1 • “past the dog”: 1 • “the dog the”: 1 • “the cat was”: 1 • “cat was quick”: 1

4.2 Understanding N-grams

Q2 — Probability Calculations

P(“ran”|“dog”) Probability that “ran” follows “dog”

$$P(\text{ran}|\text{dog}) = \frac{\text{Count}(\text{dog ran})}{\text{Count}(\text{dog})} = \frac{1}{3} = 0.33$$

P(cat|The“) Probability that “cat” follows “The”

$$P(\text{cat}|\text{The}) = \frac{\text{Count}(\text{the cat})}{\text{Count}(\text{the})} = \frac{1}{4} = 0.25$$

P(high|jumped) Probability that “high” follows “jumped”

$$P(\text{high}|\text{jumped}) = \frac{\text{Count}(\text{jumped high})}{\text{Count}(\text{jumped})} = \frac{1}{1} = 1.0$$

4.2 Understanding N-grams

To determine most likely word to follow “The dog” (using bigram model), we check the counts of “the dog X” for all X:

- $\text{Count}(\text{“the dog ran”}) = 1$
- $\text{Count}(\text{“the dog jumped”}) = 1$
- $\text{Count}(\text{“the dog the”}) = 1$

So there's a tie between with equal probability of 0.333 each ($\frac{1}{3}$).

4.2 Understanding N-grams

Probability of the sequence “The cat ran” Using the bigram model and chain rule:

$$P(\text{The cat ran}) = P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{ran}|\text{cat}) = \left(\frac{4}{18}\right) \times \left(\frac{1}{4}\right) \times \left(\frac{1}{2}\right) = 0.028$$

Where:

- $P(\text{the}) = \frac{4}{18}$ (frequency of “the” out of all words)
- $P(\text{cat}|\text{the}) = \frac{1}{4}$ (frequency of “the cat” out of all bigrams starting with “the”)
- $P(\text{ran}|\text{cat}) = \frac{1}{2}$ (frequency of “cat ran” out of all bigrams starting with “cat”)

4.2 Understanding N-grams

Danger

In n-gram language models, we estimate the probability of word sequences. However, natural sentences have boundaries, and a model must know:

1. Where a sentence begins
2. Where a sentence ends

Without explicit delimiters, the model cannot correctly estimate probabilities for:

- the first word
- the last word
- transitions involving sentence boundaries

Success

We introduce special tokens:

- `<s>` start of sentence (SOS)
- `</s>` end of sentence (EOS)

4.2 Understanding N-grams

Task 8: Estimate Bigram Probabilities (1st example)

Bigram (Markov) Approximation

The **bigram approximation** (first-order Markov assumption) simplifies each conditional to depend only on the immediately preceding word:

$$P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

4.2 Understanding N-grams

`<s>` I love NLP `</s>`

`<s>` I love machine learning `</s>`

`<s>` I enjoy NLP `</s>`

Vocabulary: `<s>`, I, love, enjoy, NLP, machine, learning, `</s>`

Total sentences = 3

4.2 Understanding N-grams

Observed Bigrams

Bigram	Count
<s> I	3
I love	2
I enjoy	1
love NLP	1
love machine	1
enjoy NLP	1
NLP </s>	2
machine learning	1
learning </s>	1

Word Counts (Denominators)

Word	Count
<s>	3
I	3
love	2
enjoy	1
NLP	2
machine	1
learning	1
</s>	3

4.2 Understanding N-grams

Bigram Probabilities

$$P(I \mid \langle s \rangle) = \frac{3}{3} = \mathbf{1.0}$$

$$P(\text{love} \mid I) = \frac{2}{3} \approx \mathbf{0.67}$$

$$P(\text{enjoy} \mid I) = \frac{1}{3} \approx \mathbf{0.33}$$

$$P(\text{NLP} \mid \text{love}) = \frac{1}{2} = \mathbf{0.5}$$

$$P(\text{machine} \mid \text{love}) = \frac{1}{2} = \mathbf{0.5}$$

$$P(\langle /s \rangle \mid \text{NLP}) = \frac{2}{2} = \mathbf{1.0}$$

⟨s⟩ I love NLP ⟨/s⟩

$$\begin{aligned} P &= P(I \mid \langle s \rangle) \times P(\text{love} \mid I) \times P(\text{NLP} \mid \text{love}) \times P(\langle /s \rangle \mid \text{NLP}) \\ &= 1 \times 0.67 \times 0.5 \times 1 \\ &\approx \mathbf{0.335} \end{aligned}$$

4.2 Understanding N-grams

Task 9: Estimate Bigram Probabilities (2nd example)

`<s>` i eat breakfast every morning `</s>`
`<s>` she eats oranges and apples every day `</s>`
`<s>` i drink coffee in the morning `</s>`
`<s>` he eats breakfast quickly `</s>`

i Info

The tokens eat and eats are considered the same.

4.2 Understanding N-grams

Q1 — Maximum-Likelihood Estimation of Bigram Probabilities

The MLE formula for a bigram probability is:

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}w_i)}{\text{Count}(w_{i-1})}$$

where $\text{Count}(\cdot)$ denotes corpus count. Compute the following:

- $P(\text{eat} | i)$
- $P(\text{apples} | \text{eat})$
- $P(\text{</s>} | \text{apples})$

4.2 Understanding N-grams

Counts from corpus:

- The corpus has two sentences beginning with `<s>` i:
 - one continuing with eat,
 - and one with drink.
- eat appears 3 times total.
- $P(\text{<s>} \mid \text{apples}) = \frac{0}{1} = \mathbf{0.000}$.

Bigram	Count($w_{i-1}w_i$)	Count(w_{i-1})	MLE
i → eat	1	2	0.500
eat → apples	0	3	0.000
apples → <code></s></code>]	0	1	0.000

4.2 Understanding N-grams

Q2 — Compute $P(\text{<s> i eat apples </s>})$ using the bigram model.

$$\begin{aligned} P &= P(i \mid \text{<s>}) \\ &\times P(\text{eat} \mid i) \\ &\times P(\text{apples} \mid \text{eat}) \\ &\times P(\text{</s>} \mid \text{apples}) \end{aligned}$$

Each factor is estimated by MLE from the corpus.

Danger

Because $P(\text{apples} \mid \text{eat}) = 0$ and $P(\text{</s>} \mid \text{apples}) = 0$, backoff is needed.

4.2 Understanding N-grams

Stupid Backoff / Unigram Fallback

When a bigram count is zero, a simple **backoff** strategy falls back to the unigram probability:

$$P_{\text{backoff}}(w_i) = \frac{\text{Count}(w_i)}{N}$$

where N is the total number of word tokens in the corpus (*excluding sentence boundary markers, or including them depending on convention*).

4.2 Understanding N-grams

Q3 — Backoff

For the target sentence `<s> i eat apples </s>`:

Zero-count bigram	Backoff to
$P(\text{apples} \mid \text{eat}) = 0$	$P(\text{apples}) = \frac{\text{Count}(\text{apples})}{N}$
$P(\text{</s>} \mid \text{apples}) = 0$	$P(\text{</s>}) = \frac{\text{Count}(\text{</s>})}{N}$

The corpus contains 4 sentences. Counting all tokens including `<s>` and `</s>`:

- Total tokens $N = 4 \times 2 + 22 = \mathbf{30}$ (4 `<s>`, 4 `</s>`, 22 content words)
- $\text{Count}(\text{apples}) = 1 \rightarrow P(\text{apples}) = 1/30 \approx 0.03$
- $\text{Count}(\text{</s>}) = 4 \rightarrow P(\text{</s>}) = 4/30 \approx 0.13$

4.2 Understanding N-grams

Q4 - Full Sentence Probability with Backoff

Combining MLE bigrams for observed bigrams and unigram backoff for zero-count bigrams:

$$P(\langle s \rangle \text{ i eat apples } \langle /s \rangle) \approx 0.500 \times 0.500 \times 0.03 \times 0.13 \approx \mathbf{0.000975}$$

Factor	Source	Value
$P(i \mid \langle s \rangle)$	MLE: $\frac{\text{Count}(\langle s \rangle, i)}{\text{Count}(\langle s \rangle)} = 2/4$	0.500
$P(\text{eat} \mid i)$	MLE: $1/2$	0.500
$P(\text{apples} \mid \text{eat})$	Backoff: $1/30$	0.03
$P(\langle /s \rangle \mid \text{apples})$	Backoff: $4/30$	0.13

4.2 Understanding N-grams

i Info

Handling Unseen **N-grams** (*Bonus*)

Add-one (Laplace) smoothing Add 1 to all counts to ensure no zero probabilities

$$P(w_2|w_1) = \frac{\text{Count}(w_1, w_2) + 1}{\text{Count}(w_1) + V}$$

Where V is the vocabulary size (number of unique words)

Add-k smoothing Add a small k ($0 < k < 1$) to all counts

Good-Turing smoothing Probability estimation is based on frequency of rare events.

Backoff models If a higher-order **n-gram** isn't observed, fall back to a lower-order **n-gram**

Interpolation Combine probabilities from different order **n-grams** with weights

4.3 Complete Pipeline Example



Text Processing

[Codes/text_processing.py](#)

4.4 Text Visualization Concepts

4.4.1 Frequency Distribution Plots

Charts showing how often different words appear in a text.

- Comparing vocabulary usage across different authors;
- Analyzing social media hashtag popularity over time;
- Studying language patterns in different genres of literature.

4.4.2 Word Clouds

A visual representation where word size corresponds to its frequency in the text.

- Analyzing customer reviews to identify common themes;
- Visualizing key topics in political speeches;
- Summarizing survey responses.

4.4 Text Visualization Concepts



Text Visualization

[Codes/text_visualization.py](#)

4.5 Common Use Cases and Applications

1. **Sentiment Analysis**

- Customer review processing
- Social media monitoring
- Brand reputation tracking

2. **Content Classification**

- News article categorization
- Spam detection
- Document sorting

3. **Text Summarization**

- News article summarization
- Document abstract generation
- Meeting notes condensation

4. **Keyword Extraction**

- SEO optimization
- Content tagging
- Research paper indexing

4.6 Best Practices and Tips

1. Choose the Right Tools

- Use NLTK for research and experimentation
- Use spaCy for production environments
- Use scikit-learn for machine learning integration

2. Performance Optimization

3. Error Handling

4. Evaluation Metrics

5. Word Embedding

5. Word Embedding

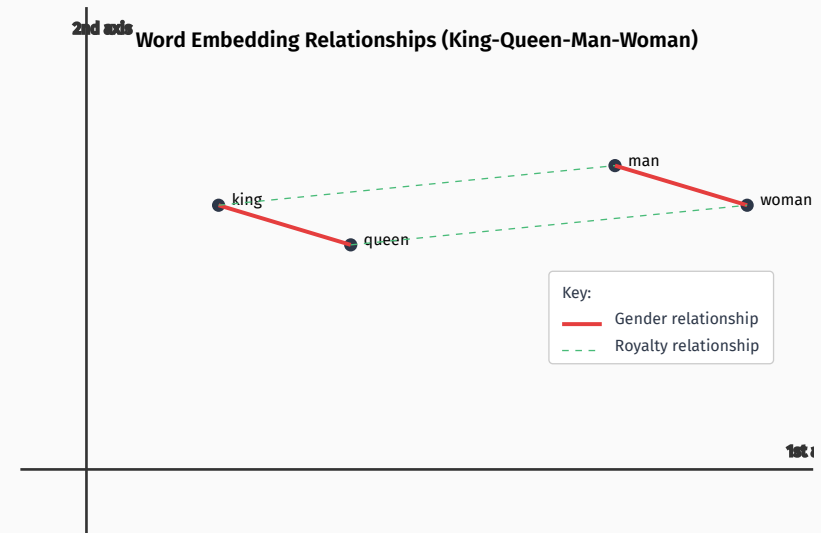
Word embedding is a technique that represents words as dense vectors of real numbers in a continuous vector space. Words with similar meanings are placed close to each other in this space. It captures semantic relationships between words.

For example, the embedding can learn that:

1 $\text{king} - \text{man} + \text{woman} \approx \text{queen}$

Dimension Reduction Unlike sparse one-hot encodings, embeddings use lower-dimensional vectors (e.g., 100-300 dimensions)

Context Awareness Embeddings are learned from large text corpora, so they reflect word usage and context.



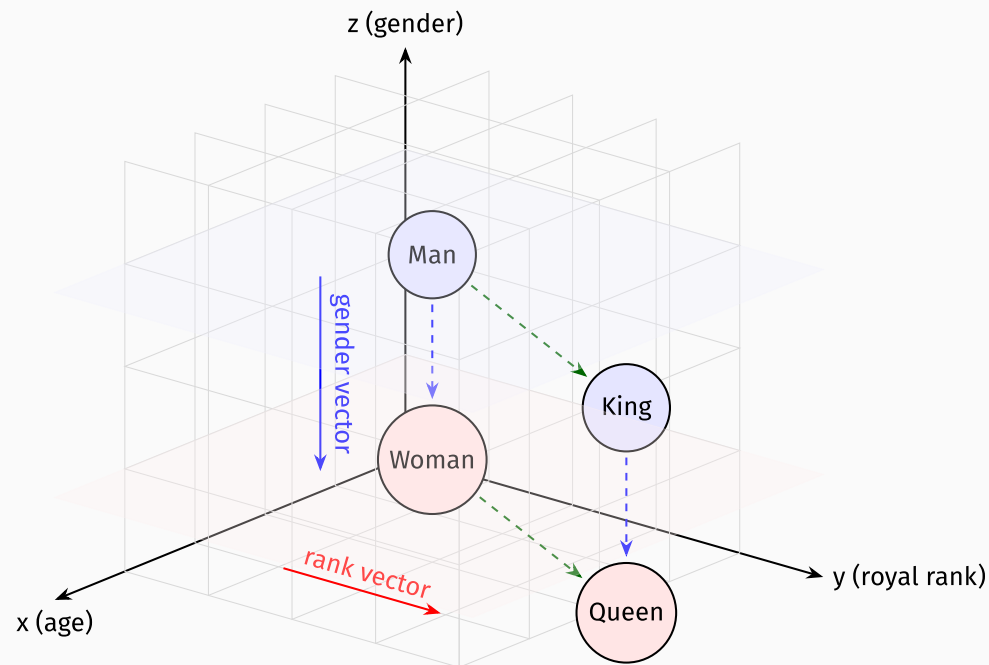
5. Word Embedding

Given these words:

```
1 ["king", "queen", "man",  
  "woman"]
```

They might be embedded as:

```
1 king → [0.7, 0.1, ... ]  
2 queen → [0.68, 0.11, ... ]  
3 man → [0.6, -0.2, ... ]  
4 woman → [0.58, -0.19, ... ]
```



5. Word Embedding

These vectors preserve semantic relationships and can be used to perform reasoning, classification, translation, and even analogy tasks in a meaningful way. For example, if we want to find the word that is to “Italy” as “Paris” is to “France”, we can use the embedding vectors:

```
1 ["Paris", "France", "Rome", "Italy"]
```

In a well-trained word embedding space, the model learns not just word meanings but also analogical relationships. It can capture that:

```
1 Paris - France + Italy ≈ Rome
```

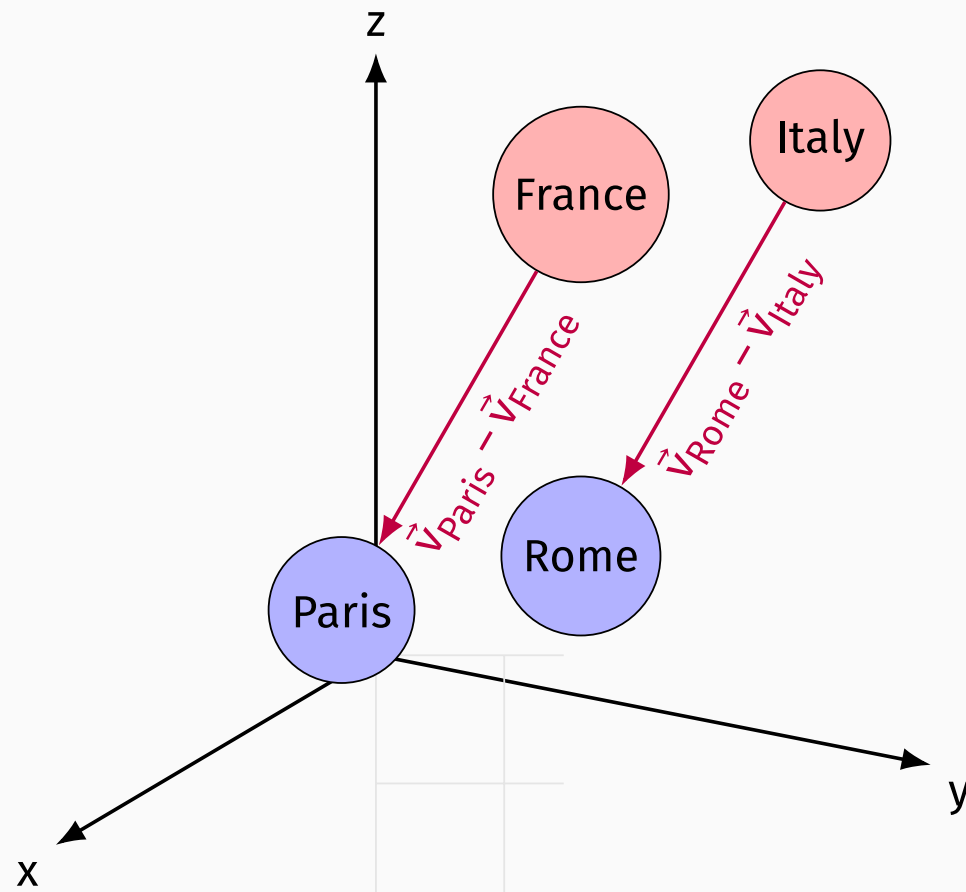
This means that the difference between “Paris” and “France” is similar to the difference between “Rome” and “Italy” — they are both capital-country pairs.

5. Word Embedding

```
1 Paris → [0.5, 0.1, 0.6, ... ]
2 France → [0.4, 0.2, 0.5, ... ]
3 Rome → [0.52, 0.12, 0.62, ... ]
4 Italy → [0.42, 0.22, 0.52, ... ]
```

If we compute:

```
1 Paris - France + Italy
2   ≈ [0.5-0.4+0.42,
3       0.1-0.2+0.22,
4       0.6-0.5+0.52]
5   = [0.52, 0.12, 0.62]
6   ≈ Rome
```



5.1 Types of Word Embedding Models

Models like Word2Vec, GloVe, and FastText are commonly used to generate these embeddings.

Word2Vec Introduced by Google; Predicts surrounding words given a target word (CBOW) or vice versa (Skip-gram).

GloVe (Global Vectors) Developed by Stanford; leverages global word co-occurrence statistics.

FastText Developed by Facebook; includes subword information to handle rare and morphologically complex words.

5.1 Types of Word Embedding Models

5.1.1 Word2Vec

Two variants

1. **Skip-Gram**: Predicts context words from a target word.
 2. **CBOW (Continuous Bag-of-Words)**: Predicts target word from context.
- Sliding window captures local context relationships.
 - Captures semantic/grammatical analogies (*e.g.*, $\text{king} - \text{man} + \text{woman} \approx \text{queen}$).
 - Lightweight and scalable for large datasets.
 - Excels at contextual similarity tasks (*e.g.*, *word clustering*).
 - Pre-trained embeddings (*e.g.*, *Google News, 300D*) for plug-and-play NLP.
 - Foundational for advanced models (*RNNs, Transformers*).

5.1 Types of Word Embedding Models

5.1.2 GloVe (Global Vectors)

- Hybrid approach: Combines **global corpus co-occurrence statistics** with **local context patterns**.
- Weighted loss function reduces impact of rare/frequent word pairs.
- Precomputed co-occurrence matrix speeds up training.
- Captures linear relationships (e.g., king - man + woman $\approx$ queen).
- Pre-trained embeddings (e.g., *Wikipedia*, *Common Crawl*, 50D-300D).
- Outperforms Word2Vec on semantic similarity tasks.
- Scalable, interpretable, and efficient for large corpora.

5.1 Types of Word Embedding Models

5.1.3 FastText

- Extends Word2Vec by incorporating *subword information* (character n-grams).
- Represents words as a bag of character n-grams (e.g., “apple” → “ap”, “app”, “pple”).
- Uses **skip-gram** or **CBOW** with subword embeddings.
- Handles rare/out-of-vocabulary (**OOV**) words via subword components.
- Robust for morphologically rich languages (e.g., *Turkish, Finnish*).
- Pre-trained vectors for 157+ languages (e.g., *Wikipedia, Common Crawl*).
- Better at handling typos, slang, and rare words.
- Strong performance in text classification and low-resource languages.
- Beats **Word2Vec** in semantic tasks
- Scalable, interpretable, integrates into **NLP** pipelines

5.2 Gensim Text Processing

5.2.1 What is Gensim?

Gensim is a robust, efficient library for topic modeling, document indexing, and similarity retrieval with large corpora. The name **Gensim** stands for “Generate Similar” - reflecting its core functionality of finding similar documents.

Key features:

- Memory efficient processing of large text collections
- Built-in implementations of popular algorithms like **Word2Vec**, **Doc2Vec**, **FastText**
- Streamlined document similarity calculations
- Topic modeling capabilities (**LSA**, **LDA**)

5.2 Gensim Text Processing

5.2.2 Basic Gensim Usage

5.2.2.1 Installation and Usage

```
1 pip install gensim
```

 Bash

```
1 import gensim
2 from gensim import corpora, models
```

 Python

5.2 Gensim Text Processing



Building a Complete Text Analysis Pipeline

[Codes/gensim_text_processing.py](#)

5.2 Gensim Text Processing

5.2.3 Best Practices and Tips

1. **Preprocessing**

- Convert to lowercase
- Remove stop words
- Apply lemmatization
- Handle special characters

5.2 Gensim Text Processing

2. Memory Efficiency

- Use streaming corpus for large datasets
- Implement memory-efficient iterators

```
1 class MyCorpus:
2     def __iter__(self):
3         for line in open('mycorpus.txt'):
4             yield dictionary.doc2bow(line.lower().split())
```

 Python

5.2 Gensim Text Processing

3. Model Persistence

Save models

```
1 dictionary.save('dictionary.gensim')  
2 tfidf_model.save('tfidf.gensim')
```

 Python

Load models

```
1 dictionary = corpora.Dictionary.load('dictionary.gensim')  
2 tfidf_model = models.TfidfModel.load('tfidf.gensim')
```

 Python

6. Deep Learning for NLP

6.1 Recurrent Neural Network (RNN)

- A type of neural network designed for sequential data
- Maintains a hidden state that captures information from previous inputs
- Suitable for language where order matters
- Processes input one token at a time
- Updates hidden state at each step

$$h_t = f(W_x x_t + W_h h_{t-1} + b_h)$$

6.1 Recurrent Neural Network (RNN)

Why RNNs for NLP?

- Captures temporal dependencies
- Handles variable-length sequences

Input sentence: "I love NLP"

Processing:

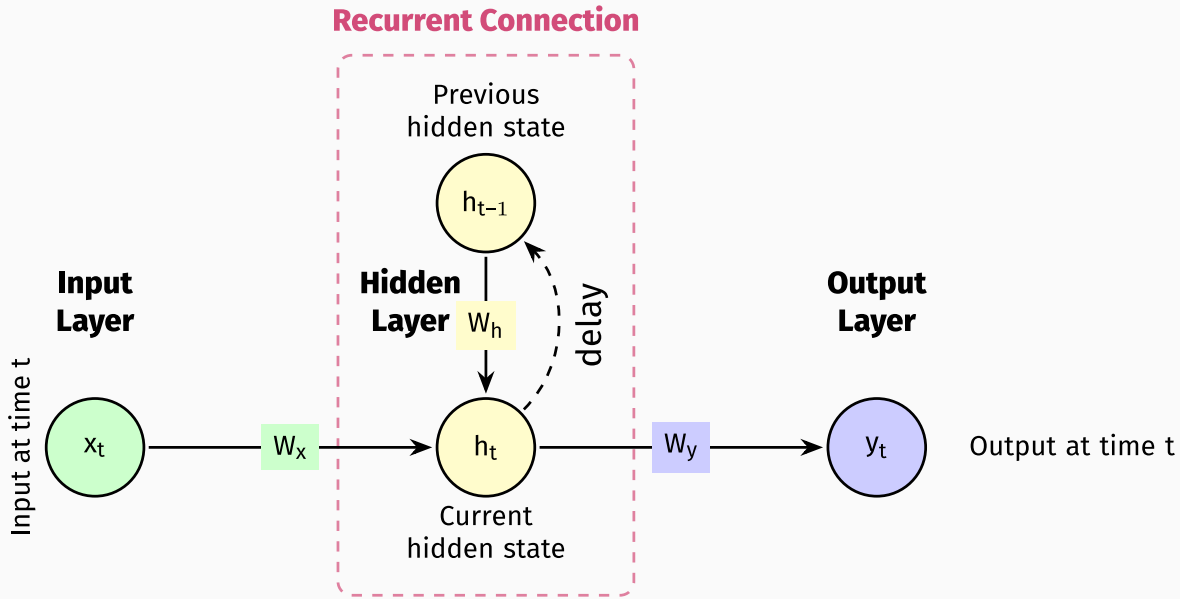
Step 1 "I" $\rightarrow h_1$

Step 2 "love" $\rightarrow h_2$

Step 3 "NLP" $\rightarrow h_3$

Each state carries context forward

6.1 Recurrent Neural Network (RNN)



Legend:

- Input Nodes
- Hidden State
- Output Nodes

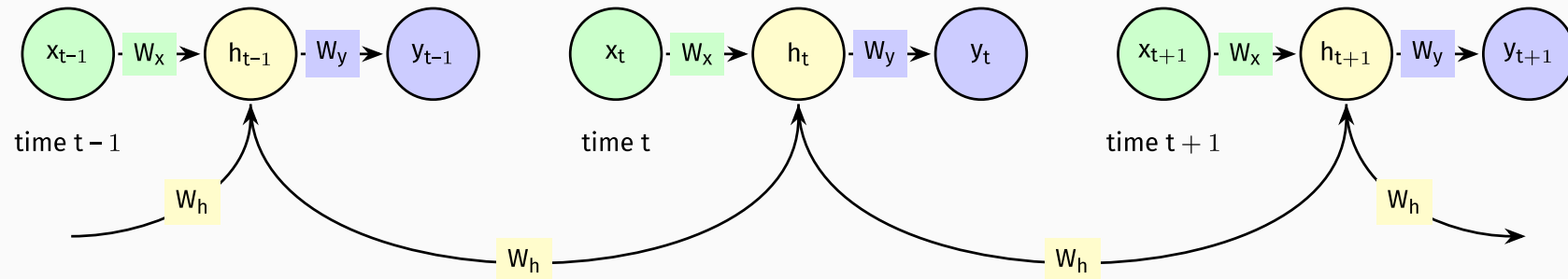
RNN Equations:

Hidden state: $h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$

Output: $y_t = f(W_y \cdot h_t + b_y)$

- *tanh or ReLU for hidden layer,*
- *softmax/sigmoid for output*

Unfolded RNN (Through Time)



6.1 Recurrent Neural Network (RNN)

- Processes sequential data
- Shares parameters across time
- Memory of previous inputs
- Useful for text, speech, time series
- Struggles with long-term dependencies
- Prone to vanishing/exploding gradients
- Suitable for tasks like sentiment analysis, text classification, and sequence labeling.

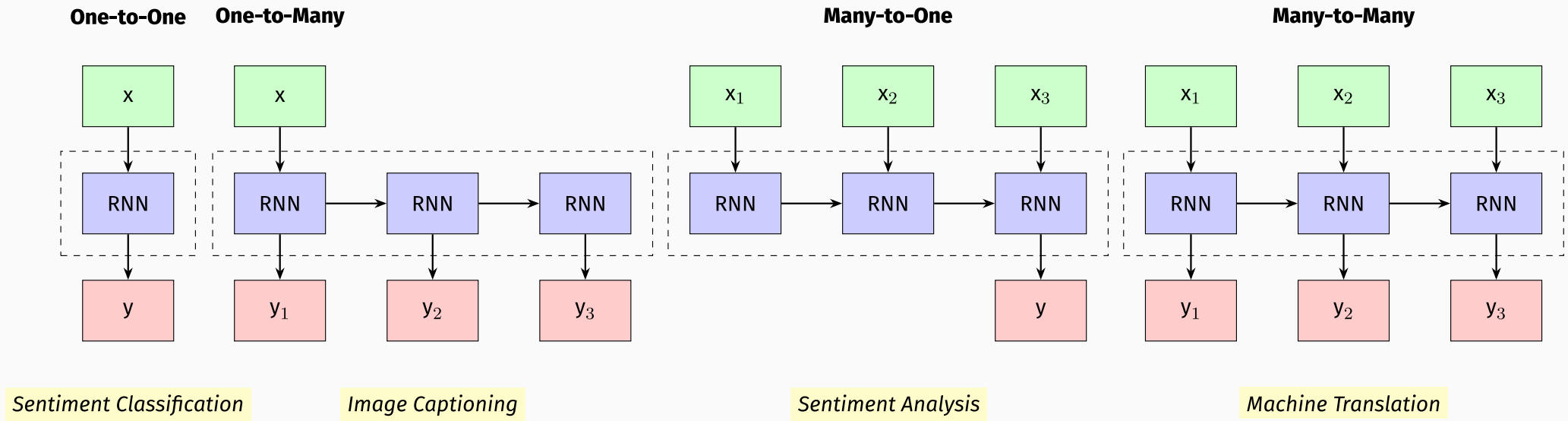
6.1 Recurrent Neural Network (RNN)

One-to-One A standard **RNN** that takes a single input and produces a single output

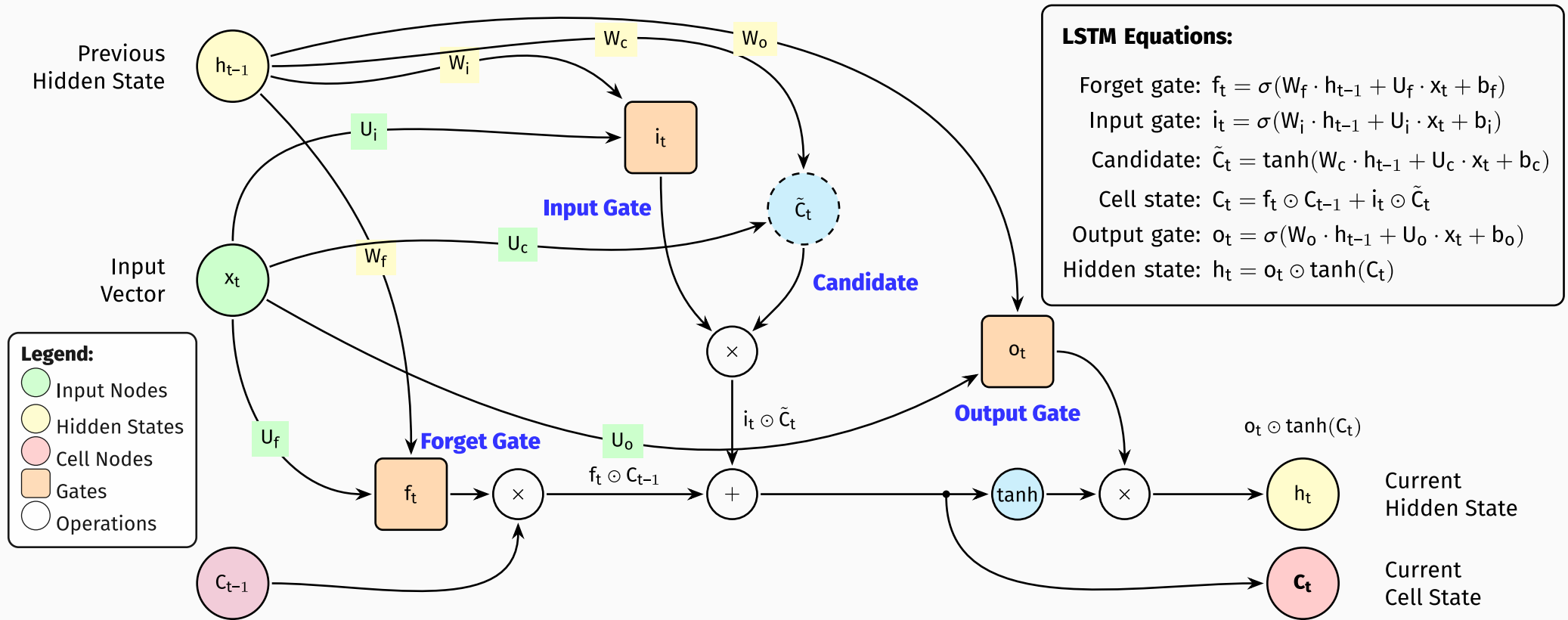
One-to-Many Takes a single input and produces a sequence of outputs

Many-to-One Takes a sequence of inputs and produces a single output

Many-to-Many Takes a sequence of inputs and produces a sequence of outputs



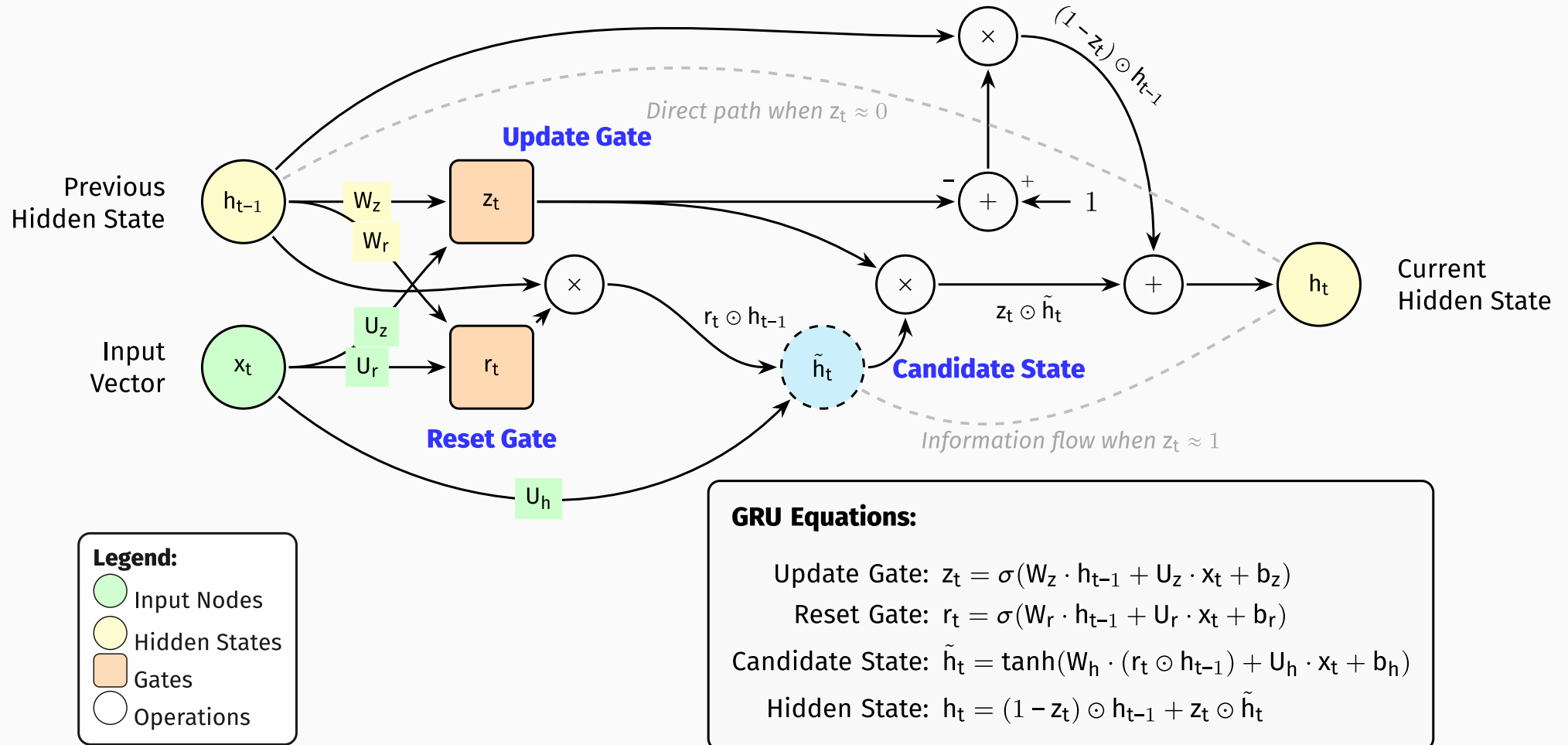
6.2 Long Short-Term Memory (LSTM)



6.2 Long Short-Term Memory (LSTM)

- Extends **RNN** by adding a cell state
- Uses gates (*input, forget, output*) to control information flow
- Better at capturing long-term dependencies
- Uses a cell state to carry information across time steps
- Can be slower to train due to the complexity of the architecture
- Can be computationally expensive for long sequences.

6.3 Gated Recurrent Unit (GRU)



6.3 Gated Recurrent Unit (GRU)

- A simplified version of **LSTM**.
- Uses update gate (z_t) to control information flow
- Uses reset gate (r_t) to control the hidden state
- Combines input and forget gates into a single update gate
- Less complex than **LSTM**, making it easier to implement and understand
- Can be faster to train due to fewer parameters.

6.3 Gated Recurrent Unit (GRU)

RNN vs LSTM vs GRU

- **RNNs** are suitable for simpler tasks with shorter sequences, but struggle with long-term dependencies
- **LSTMs** are more complex but better at capturing long-term dependencies
- **GRUs** are a middle ground, offering a simpler architecture with comparable performance to **LSTMs**
- **LSTMs** and **GRUs** are often used in applications where long-term dependencies are crucial, such as language modeling and text generation.

6.3 Gated Recurrent Unit (GRU)



Sequence Processing Foundations

[Jupyter/sequence_processing.ipynb](#)

6.4 Transformer

The **Transformer** is a neural network architecture that has become the foundation of most modern NLP models. Introduced in 2017 by Vaswani et al. in the paper “**Attention Is All You Need**”¹, it replaced earlier sequence models like **RNN** and **LSTM** by introducing a fully attention-based mechanism.

Transformers have significantly advanced the field of NLP by enabling models to learn deeper context, handle longer dependencies, and train faster due to parallel computation. Unlike **RNN**, which process input sequentially, **Transformers** can process all tokens at once. This allows faster training and better scalability on large datasets.

¹<https://arxiv.org/abs/1706.03762>

6.4 Transformer

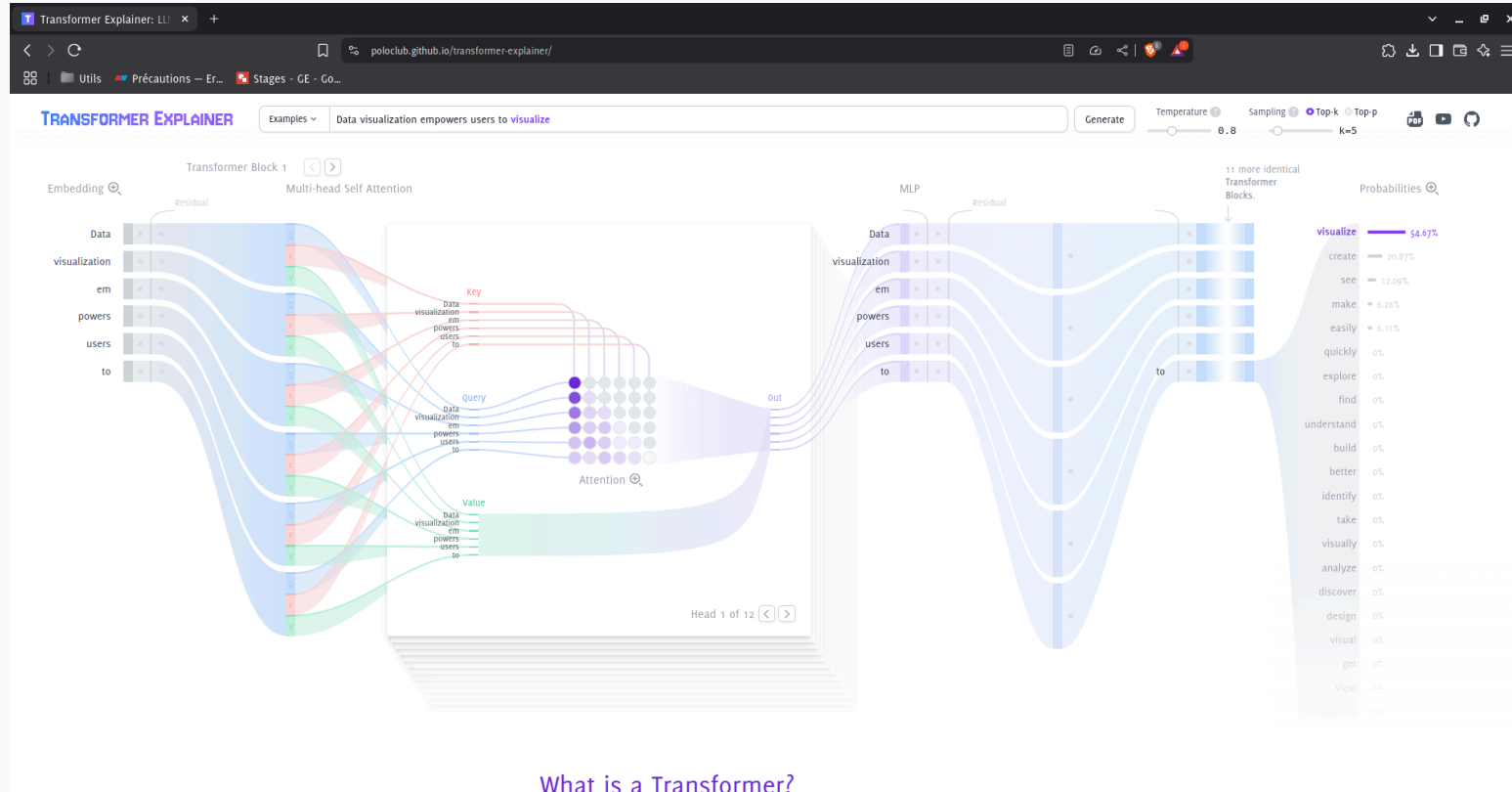


Figure 5: Transformer explained visually¹.

¹<https://poloclub.github.io/transformer-explainer/>

6.4 Transformer

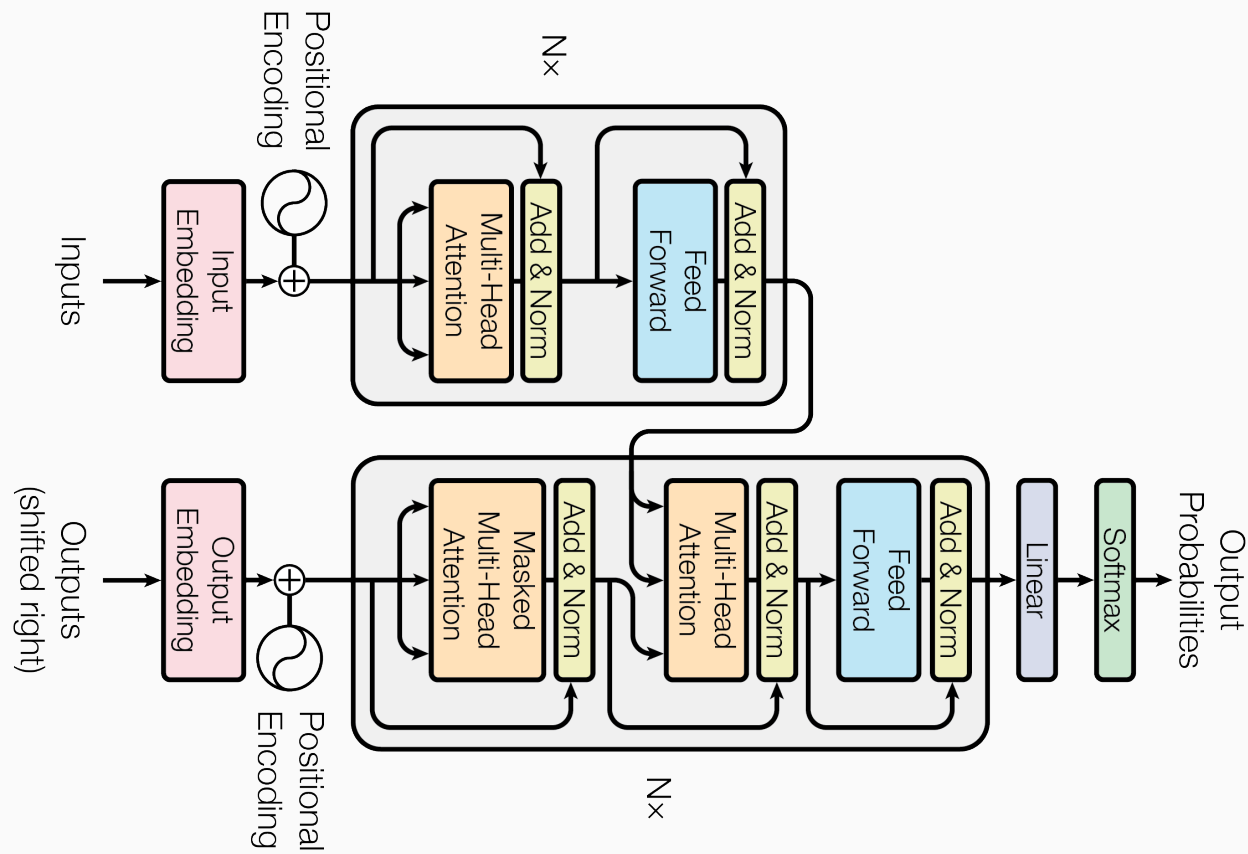


Figure 6: Core Components¹.

¹<https://arxiv.org/abs/1706.03762>

6.4 Transformer

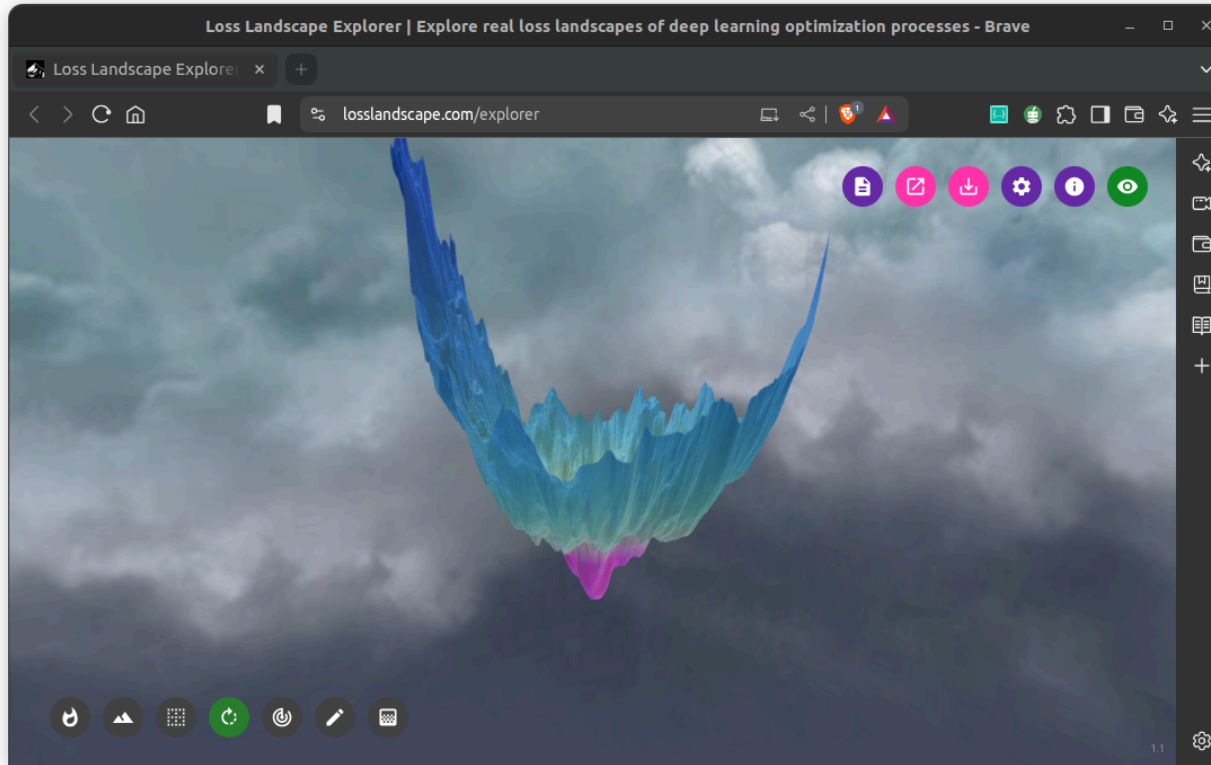


Figure 7: Why Skip Connection¹?

¹<https://losslandscape.com/explorer>

6.4.1 Positional Encoding

- Transformers don't inherently understand word order
- Positional encodings are added to word embeddings to inject information about the position of words in a sentence.

6.4 Transformer

Task 10:

We have a short sentence I like to learn. The initial learned embeddings for these words are:

$$E(\text{"I"}) = [0.1, 0.2, 0.1, -0.3, 0.5]$$

$$E(\text{"like"}) = [-0.3, 0.4, 0.6, -0.2, 0.1]$$

$$E(\text{"to"}) = [0.5, -0.1, 0.0, 0.3, 0.2]$$

$$E(\text{"learn"}) = [0.2, 0.6, -0.4, 0.1, 0.7]$$

1. Calculate the PE vector for the word "like", which is at position $\text{pos} = 1$.
2. Calculate the PE vector for the word "learn", which is at position $\text{pos} = 3$.

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \text{ and } \text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

6.4 Transformer

1. “like” at pos = 1:

$$i = 0 \Rightarrow \text{PE}(1, 0) = \sin\left(\frac{1}{10000^{\frac{0}{5}}}\right) \approx 0.8415$$

$$i = 0 \Rightarrow \text{PE}(1, 1) = \cos\left(\frac{1}{10000^{\frac{0}{5}}}\right) \approx 0.5403$$

$$i = 1 \Rightarrow \text{PE}(1, 2) = \sin\left(\frac{1}{10000^{\frac{2}{5}}}\right) \approx 0.0251$$

$$i = 1 \Rightarrow \text{PE}(1, 3) = \cos\left(\frac{1}{10000^{\frac{2}{5}}}\right) \approx 0.9997$$

$$i = 2 \Rightarrow \text{PE}(1, 4) = \sin\left(\frac{1}{10000^{\frac{4}{5}}}\right) \approx 0.0006$$

2. “learn” at pos = 3:

$$i = 0 \Rightarrow \text{PE}(3, 0) = \sin\left(\frac{3}{10000^{\frac{0}{5}}}\right) \approx 0.1411$$

$$i = 0 \Rightarrow \text{PE}(3, 1) = \cos\left(\frac{3}{10000^{\frac{0}{5}}}\right) \approx -0.9900$$

$$i = 1 \Rightarrow \text{PE}(3, 2) = \sin\left(\frac{3}{10000^{\frac{2}{5}}}\right) \approx 0.0753$$

$$i = 1 \Rightarrow \text{PE}(3, 3) = \cos\left(\frac{3}{10000^{\frac{2}{5}}}\right) \approx 0.9972$$

$$i = 2 \Rightarrow \text{PE}(3, 4) = \sin\left(\frac{3}{10000^{\frac{4}{5}}}\right) \approx 0.0019$$

Positional Encoding Calculations

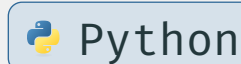
Word	Position (pos)	PE Vector (<i>rounded to 4 decimals</i>)
“like”	1	[0.8415, 0.5403, 0.0251, 0.9997, 0.0006]
“learn”	3	[0.1411, -0.9900, 0.0753, 0.9972, 0.0019]

Final Input Vectors (Embedding + PE)

Word	Final Input Vector (<i>rounded to 4 decimals</i>)
“like”	[0.5415, 0.9403, 0.6251, 0.7997, 0.1006]
“learn”	[0.3411, -0.3900, -0.3247, 1.0972, 0.7019]

6.4 Transformer

```
1  # Create a matrix of zeros to store positional encodings
2  pe = torch.zeros(max_seq_length, d_model)
3  # Generate positions and add a dimension for broadcasting
4  pos = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
5  # division term for the sine and cosine arguments (1 / 10000^(2i/
   d_model))
6  div_term = torch.exp(torch.arange(0, d_model, 2).float() * -
   (math.log(10000.0) / d_model))
7
8  # Apply sine to even indices in the embedding (2i)
9  pe[:, 0::2] = torch.sin(pos * div_term)
10 # Apply cosine to odd indices in the embedding (2i+1)
11 pe[:, 1::2] = torch.cos(pos * div_term)
```



Python

6.4 Transformer

max_len vs seq_len in NLP

seq_len — Sequence Length

The **actual** number of tokens in a given input after tokenization.

- Varies per sample
- “Hello world” → seq_len = 2
- “The cat sat on the mat” → seq_len = 6

max_len — Maximum Length

A **fixed upper bound** set by the model or developer. All sequences are padded or truncated to fit.

- BERT: max_len = 512
- GPT-2: max_len = 1024
- Custom model: you define it

Rule: if seq_len < max_len → pad with [PAD] tokens | if seq_len > max_len → truncate to max_len

6.4.2 Encoder-Decoder Structure

- **Encoder:** Takes the input sequence and encodes it into a context-aware representation
- **Decoder:** Uses the encoded representation to generate output (*e.g., in translation or summarization*).

6.4.3 Self-Attention

- Each word in a sentence pays attention to every other word to understand context
- This allows the model to dynamically weigh the importance of different words

In the sentence “**She poured water into the glass until it was full.**”, the model can learn that “**it**” refers to “**glass**”.



Quote

The key/value/query concept is analogous to retrieval systems. For example, when you search for videos on Youtube, the search engine will map your query (text in the search bar) against a set of keys (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (values)¹.

¹<https://stats.stackexchange.com/questions/421935/what-exactly-are-keys-queries-and-values-in-attention-mechanisms>

6.4 Transformer

Architecture

- Multi-head self-attention layers
- Feed-forward neural networks
- Layer normalization
- Residual connections

Use Cases

- Machine Translation
- Text Generation
- Question Answering
- Text Classification
- Summarization

Popular Models

- BERT
- GPT
- T5
- RoBERTa, XLNet, DistilBERT

6.4 Transformer

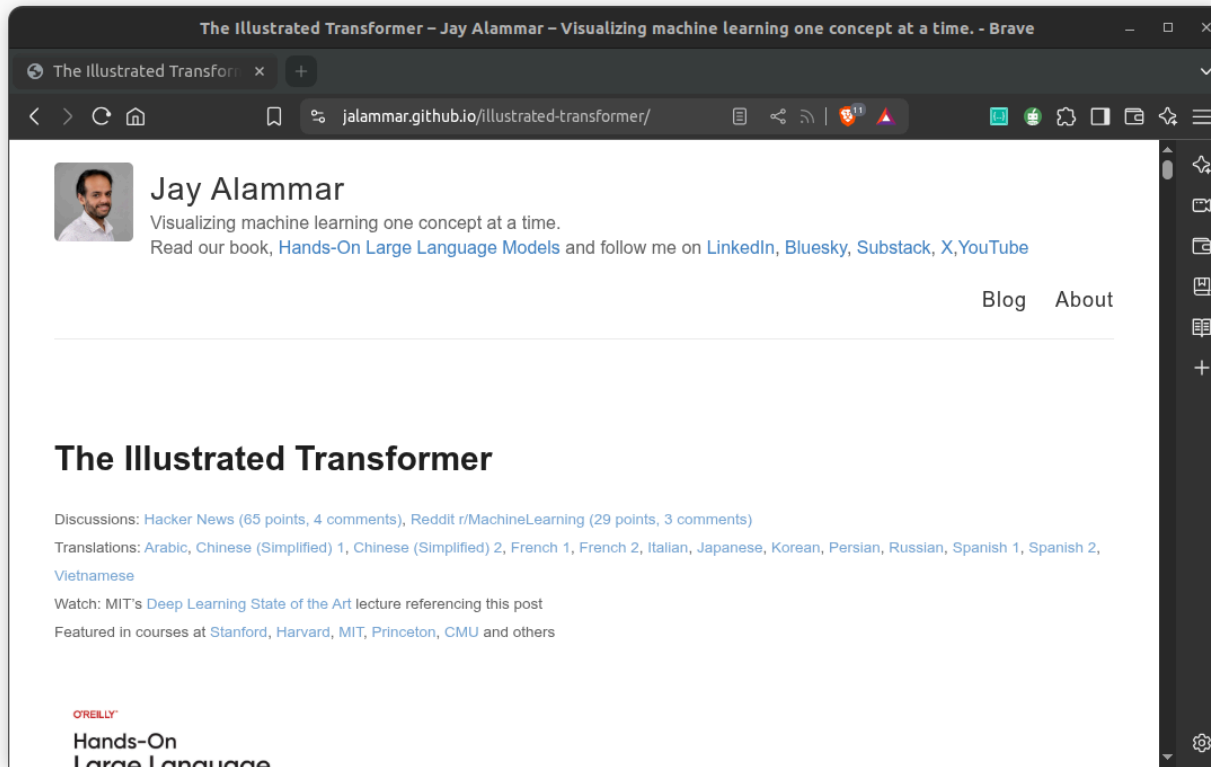
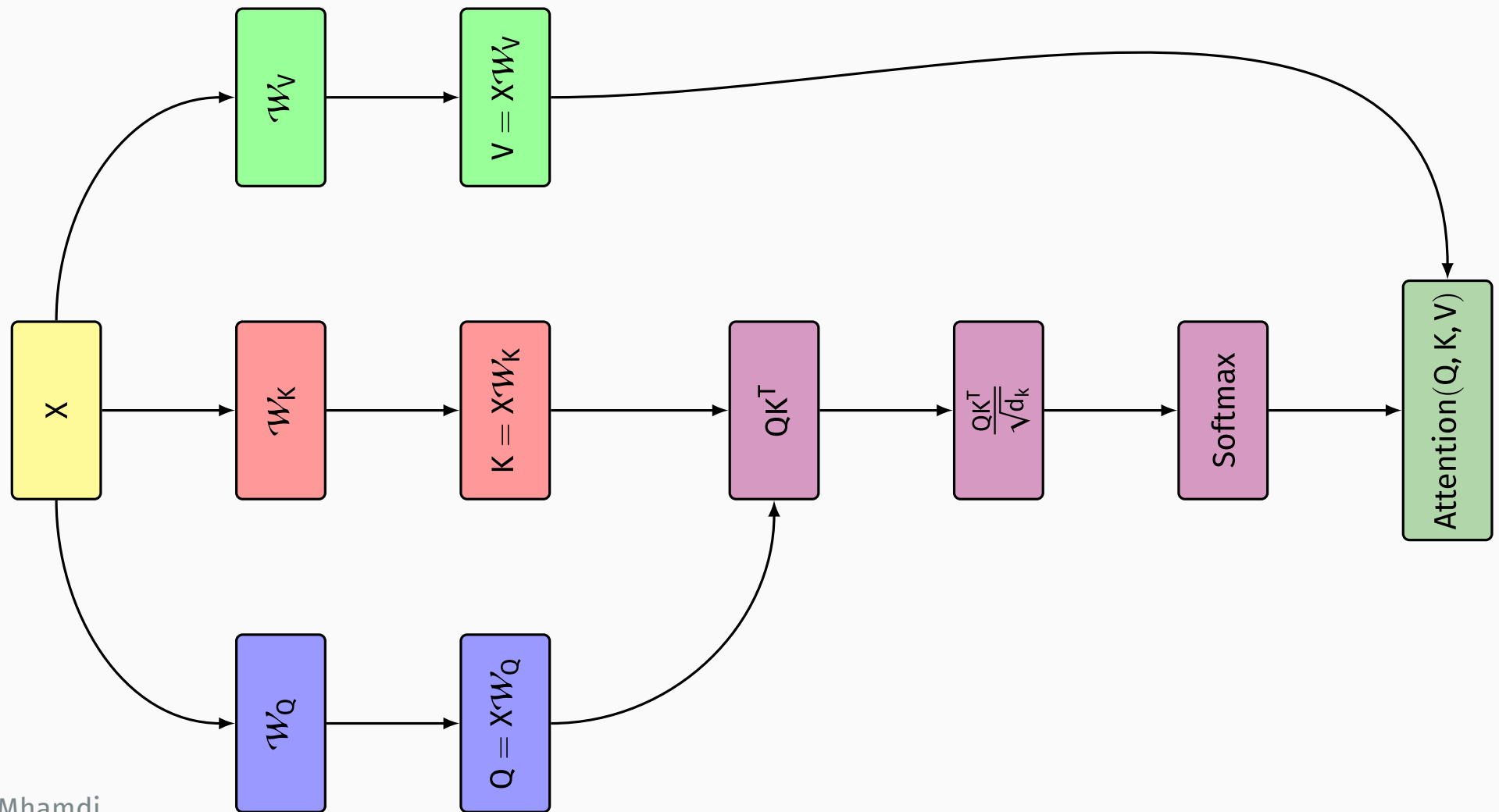


Figure 8: Transformer Illustrated¹.

¹<https://jalamar.github.io/illustrated-transformer/>

6.4 Transformer



Task 11: Self-Attention Mechanism

Consider the sentence “I like to learn” with the following setup:

- Sequence length: 4 words
 - Embedding dimension: 5
1. Compute the Query (Q), Key (K), and Value (V) matrices for all words
 2. Calculate the attention scores (QK^T) for the word “like” attending to all words
 3. Apply softmax to get attention weights for “like”
 4. Compute the weighted sum of values for “like”
 5. Repeat steps 2-4 for the word “learn”

6.4 Transformer

The learned embeddings for each word are:

$$\mathbf{E}(\text{"I"}) = [0.1, 0.2, 0.1, -0.3, 0.5]$$

$$\mathbf{E}(\text{"like"}) = [-0.3, 0.4, 0.6, -0.2, 0.1]$$

$$\mathbf{E}(\text{"to"}) = [0.5, -0.1, 0.0, 0.3, 0.2]$$

$$\mathbf{E}(\text{"learn"}) = [0.2, 0.6, -0.4, 0.1, 0.7]$$

The weight matrices are:

$$W_Q = \begin{pmatrix} 0.1 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.0 & 0.1 & 0.2 & 0.0 & 0.1 \\ 0.1 & 0.0 & 0.1 & 0.2 & 0.0 \\ 0.0 & 0.1 & 0.0 & 0.1 & 0.2 \\ 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \end{pmatrix}, W_K = \begin{pmatrix} 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \\ 0.0 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 & 0.2 & 0.0 \\ 0.1 & 0.0 & 0.1 & 0.0 & 0.2 \end{pmatrix}, W_V = \begin{pmatrix} 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 & 0.2 & 0.0 \\ 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \\ 0.0 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \end{pmatrix}$$

Use a scaling factor of $\sqrt{5} \approx 2.24$ for the attention scores.

Q1 — Compute Q, K, V matrices

First, let's organize our input matrix X (4×5):

$$X = \begin{pmatrix} 0.1 & 0.2 & 0.1 & -0.3 & 0.5 \\ -0.3 & 0.4 & 0.6 & -0.2 & 0.1 \\ 0.5 & -0.1 & 0.0 & 0.3 & 0.2 \\ 0.2 & 0.6 & -0.4 & 0.1 & 0.7 \end{pmatrix}$$

6.4 Transformer

Query Matrix $Q = XW_Q$:

- Q_1 ("I"): [0.12, 0.01, 0.10, 0.00, 0.01]
- Q_2 ("like"): [0.05, -0.04, 0.15, 0.07, 0.01]
- Q_3 ("to"): [0.09, 0.12, 0.00, 0.08, 0.07]
- Q_4 ("learn"): [0.12, 0.11, 0.15, -0.05, 0.15]

Key Matrix $K = XW_K$:

- K_1 ("I"): [0.08, 0.01, 0.08, -0.04, 0.12]
- K_2 ("like"): [0.01, 0.06, 0.10, 0.00, 0.05]
- K_3 ("to"): [0.12, 0.01, 0.07, 0.05, 0.09]
- K_4 ("learn"): [0.07, 0.13, 0.01, 0.08, 0.12]

Value Matrix $V = XW_V$:

- V_1 ("I"): [0.08, -0.04, 0.13, 0.01, 0.07]
- V_2 ("like"): [0.10, 0.00, 0.02, 0.06, 0.04]
- V_3 ("to"): [0.07, 0.05, 0.14, 0.01, 0.07]
- V_4 ("learn"): [0.01, 0.08, 0.14, 0.13, 0.05]

6.4 Transformer

Q2 — Attention scores for “like” ($Q_2 K^T$)

$$Q_2 = [0.05, -0.04, 0.15, 0.07, 0.01]$$

Attention scores (before scaling):

- $Q_2 \cdot K_1 = 0.05 \times 0.08 + (-0.04) \times 0.01 + 0.15 \times 0.08 + 0.07 \times (-0.04) + 0.01 \times 0.12 = 0.0140$
- $Q_2 \cdot K_2 = 0.05 \times 0.01 + (-0.04) \times 0.06 + 0.15 \times 0.10 + 0.07 \times 0.00 + 0.01 \times 0.05 = 0.0136$
- $Q_2 \cdot K_3 = 0.05 \times 0.12 + (-0.04) \times 0.01 + 0.15 \times 0.07 + 0.07 \times 0.05 + 0.01 \times 0.09 = 0.0205$
- $Q_2 \cdot K_4 = 0.05 \times 0.07 + (-0.04) \times 0.13 + 0.15 \times 0.01 + 0.07 \times 0.08 + 0.01 \times 0.12 = 0.0066$

Scaled attention scores ($\div \sqrt{5} \approx \div 2.24$):

$$Q_2 K^T = [0.0063, 0.0061, 0.0092, 0.0030]$$

Q3 — Softmax for “like”

Softmax calculation:

- $\exp(0.0063) = 1.0063$
- $\exp(0.0061) = 1.0061$
- $\exp(0.0092) = 1.0092$
- $\exp(0.0030) = 1.0030$

Attention weights ($Sum = 4.0246$):

$$\alpha_2 = [0.2500, 0.2500, 0.2508, 0.2492]$$

Q4 — Weighted sum of values for “like”

$$\begin{aligned}\text{Weighted } V &= 0.2500 \times V_1 + 0.2500 \times V_2 + 0.2508 \times V_3 + 0.2492 \times V_4 \\ &= 0.2500 \times [0.08, -0.04, 0.13, 0.01, 0.07] \\ &\quad + 0.2500 \times [0.10, 0.00, 0.02, 0.06, 0.04] \\ &\quad + 0.2508 \times [0.07, 0.05, 0.14, 0.01, 0.07] \\ &\quad + 0.2492 \times [0.01, 0.08, 0.14, 0.13, 0.05]\end{aligned}$$

$$\text{Weighted } V = [\mathbf{0.0650, 0.0225, 0.1075, 0.0524, 0.0575}]$$

6.4 Transformer

Q5 — Repeat for “learn” (Q_4)

$$Q_4 = [0.12, 0.11, 0.15, -0.05, 0.15]$$

Attention scores (*before scaling*):

$$\begin{aligned} \bullet Q_4 \cdot K_1 &= 0.0427 & \bullet Q_4 \cdot K_2 &= 0.0303 & \bullet Q_4 \cdot K_3 &= 0.0370 & \bullet Q_4 \cdot K_4 &= 0.0382 \end{aligned}$$

Scaled attention scores ($\div \sqrt{5} \approx \div 2.24$): $[0.0191, 0.0136, 0.0165, 0.0171]$

Attention weights: $\alpha_4 = [0.2506, 0.2492, 0.2500, 0.2501]$

Weighted sum of values for “learn”: = **$[0.0650, 0.0225, 0.1076, 0.0525, 0.0575]$**

Both “like” and “learn” produce nearly uniform attention weights (0.25 per word), resulting in output vectors that are almost identical. In a trained model, W_Q , W_K , and W_V would learn to amplify meaningful differences.



Attention Mechanisms and Advanced Architectures

[Jupyter/attention_transformers.ipynb](#)

6.5 Large Language Models (LLMs)

Large Language Models (LLMs) represent a paradigm shift in **AI**, moving beyond traditional **NLP** tasks toward more general language capabilities that increasingly resemble human-like understanding.

They are neural network models trained on vast corpora of text data using self-supervised learning techniques. Most modern **LLMs** are based on the transformer architecture, specifically using variants of the decoder-only design pioneered by **GPT (Generative Pre-trained Transformer)** models.

6.5 Large Language Models (LLMs)

6.5.1 Capabilities and Applications

Modern **LLMs** exhibit remarkable versatility across numerous **NLP** tasks:

- Text generation and creative writing
- Question answering and information retrieval
- Summarization and content transformation
- Code generation and debugging
- Translation between languages
- Reasoning about complex problems
- Instruction following and conversational abilities

6.5 Large Language Models (LLMs)

6.5.2 Core Characteristics

Scale

Training Data

Emergent Abilities

Transfer Learning

Context Window

6.5 Large Language Models (LLMs)

6.5.3 Training Paradigm

LLMs follow a training process typically divided into phases:

Pre-training

Fine-tuning

RLHF (Reinforcement Learning from Human Feedback)

6.5 Large Language Models (LLMs)

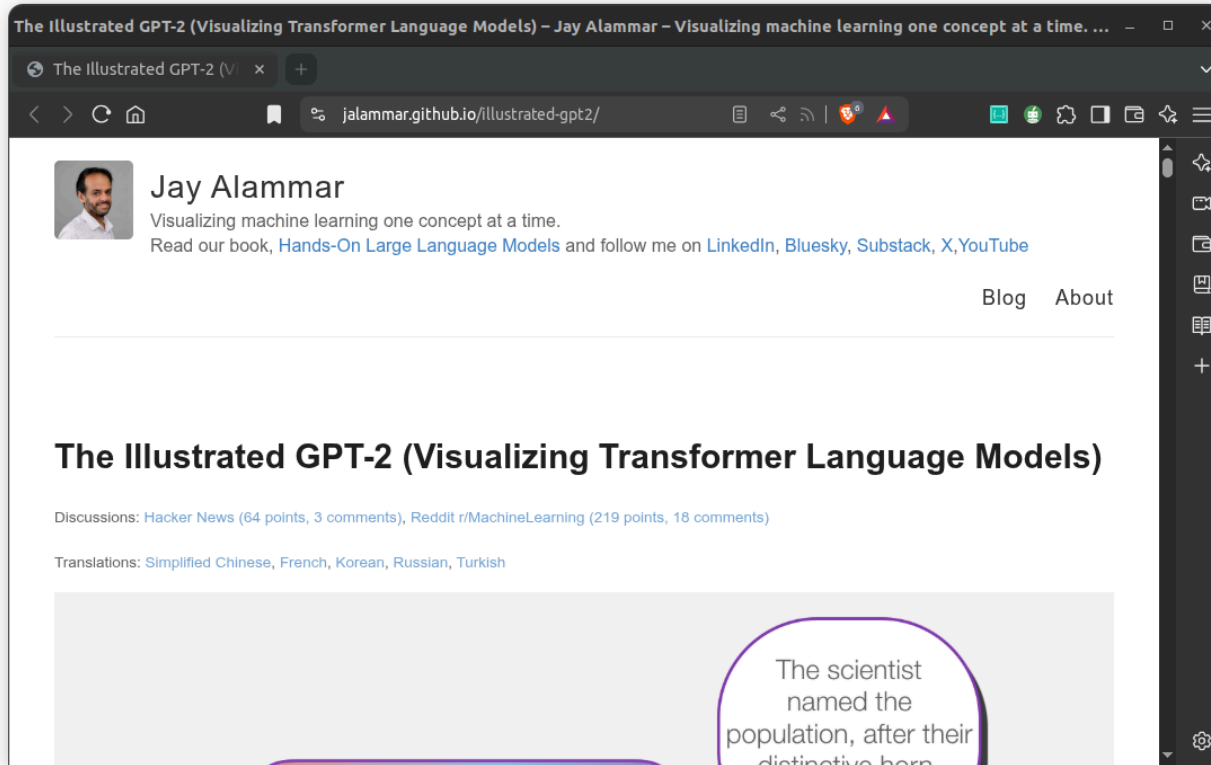


Figure 9: Illustrated GPT2¹.

¹<https://jalamar.github.io/illustrated-gpt2/>

6.5 Large Language Models (LLMs)

6.5.4 Limitations and Challenges

- **Hallucinations**
- **Reasoning limitations**
- **Training data cutoff**
- **Computational requirements**
- **Bias and toxicity**
- **Black-box nature**

6.5 Large Language Models (LLMs)

6.5.5 Examples of Prominent LLMs

Model	Organization	Year	Notable Features
GPT-4	OpenAI	2023	Multimodal capabilities, advanced reasoning
Claude	Anthropic	2023	Long context window, helpful assistant
Gemini	Google	2023	Strong performance across benchmarks
LLaMA 2	Meta	2023	Open-weights research model
Mistral	Mistral AI	2023	Efficient architecture, strong performance

6.5 Large Language Models (LLMs)

6.5.6 Model Size Comparison

Model	Parameters (Billions)	Max Context Win- dow	Notes
GPT-2	1.5	1,024	Standard dense transformer
GPT-3	175	2,048	The “Davinci” series foundation
GPT-3.5	175	4,096 – 16,384	16k available in turbo-0613
GPT-4	1,760 (MoE)	8,192 – 128,000	Uses Mixture of Experts (MoE)
Claude 2.1	150+	200,000	Industry leader in context in 2023
PaLM	540	8,192	Google’s Pathways architecture
LLaMA 2	7, 13, 70	4,096	Meta’s open-weights milestone
Gemini 1.0 Ultra	1,000+	32,768	First model to outperform GPT-4 on MMLU

6.5 Large Language Models (LLMs)

6.5.7 Future Directions

As **LLMs** continue to evolve, several promising research directions are emerging:

1. **Multimodality**
2. **Efficiency**
3. **Tool use**
4. **Memory systems**
5. **Alignment**

6.5 Large Language Models (LLMs)

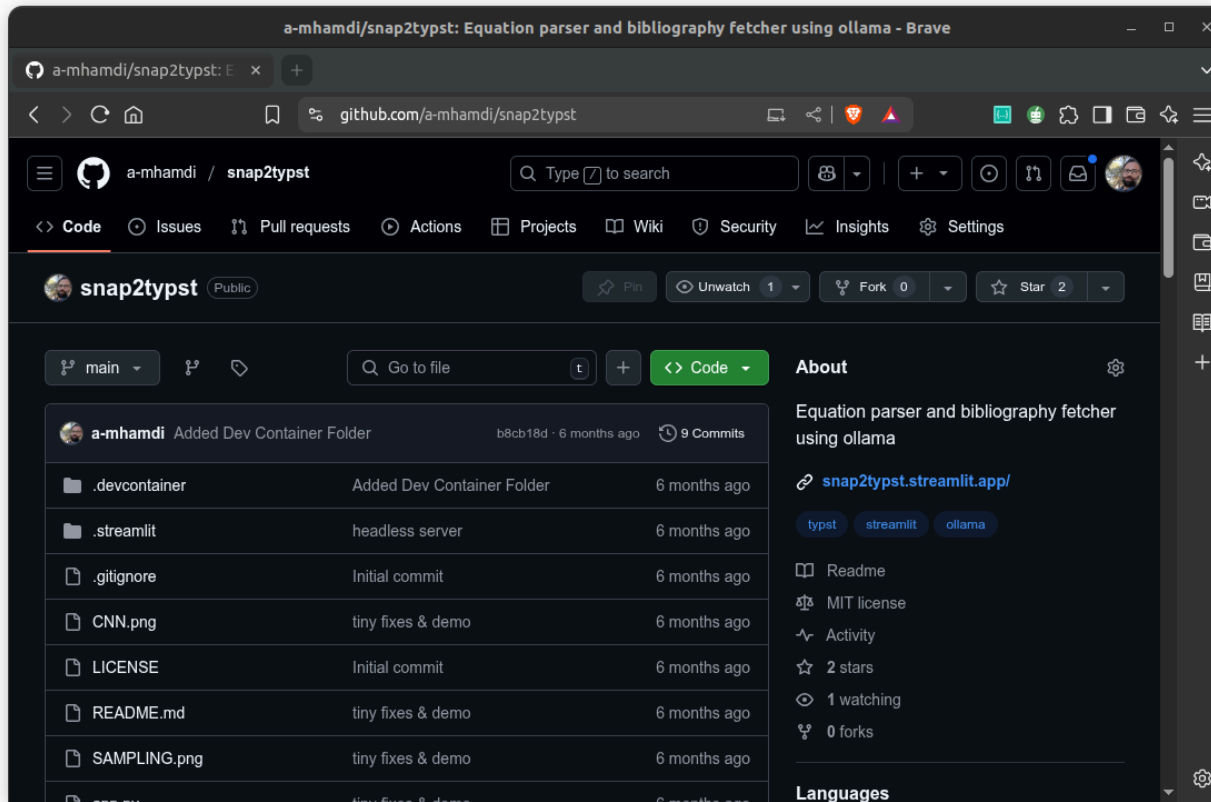


Figure 10: Equation Parser and Bibliography Fetcher for Typst¹.

¹<https://github.com/a-mhamdi/snap2typst>

Thank you for your attention

Bibliography

Bibliography

- Alammar, J., & Grootendorst, M. (2024). *Hands-on large language models: Language understanding and generation*. O'Reilly Media, Inc.
- Brown, P. F., Desouza, P. V., Mercer, R. L., Pietra, V. J. D., & Lai, J. C. (1992). Class-based n-gram models of natural language. *Computational Linguistics*, 18(4), 467–479.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
- Harris, Z. S. (1954). Distributional structure. *Word*, 10(2–3), 146–162.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength Natural Language Processing in Python*. <https://spacy.io/>
- Raschka, S. (2025). *Build a large language model (from scratch)*. Manning Publications.
- Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. 28(1), 11–21.

- Tunstall, L., Werra, L. v., & Wolf, T. (2023). *Natural language processing with transformers: Building language applications with hugging face*. O'Reilly Media.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.